

Research Report on

Scholarly metamorphic testing methods characterizing MR identification modes and operator algebraic MR class derivation

Full search query: I want to find rigorous, verifiable scholarly literature centered on metamorphic testing and metamorphic relation identification, in order to establish whether prior work has mainly treated MR identification as an expert-driven, taxonomy-based, or search/generated process, and whether any existing methods systematically derive MR classes from program-family operator algebra with explicit origin and boundary; using adjacent evidence on algebraic laws, invariants, symmetries, monotonicity, conservation, and equivalence-preserving transformations only insofar as they support this MT-centered positioning, so that NOETHER can be precisely framed as a complementary MR identification method whose main contribution is operator-algebraic derivation of MR classes and boundaries, with operator-block coverage and traceable algebraic origin as evidence dimensions distinct from fault-detection effectiveness

Summary

The literature shows that essentially all prior work on metamorphic relation (MR) identification is expert driven, taxonomy/pattern based, or search/ML/model based—with no existing framework systematically deriving MR classes and boundaries from a program family operator algebra—so NOETHER [1] appears to be the first to make operator algebraic derivation, operator block coverage, and traceable algebraic origin explicit evidence dimensions distinct from fault detection effectiveness.

Overall Landscape: How MR Identification Has Been Treated

Dominant paradigms: expert, taxonomy, search/ML, model based

Across surveys and primary papers, MR identification is consistently framed along three main lines, plus a smaller “model based” strand:

- **Expert/manual MRs**, often informed by domain knowledge, physics, specs, or code inspection.
 - Nuclear and CRAM work defines hierarchical MR models (physics/algorithm/code layers) and derives MRs from physical equations and numerical schemes [18,56].
 - Many domain studies in AI/ML and scientific computing also hand craft MRs; surveys emphasize this as the historical norm [5,6,54].
- **Taxonomy / pattern based identification** where MRs are catalogued and reused as patterns:
 - **METRIC / METRIC+ / CCML**: category choice frameworks where testers select MR categories based on input/output domain properties; METRIC+ adds output domain structure and CCML/CCMT provide a general MR description language and tool support [8,10,13].
 - **MR patterns (MRPs, MRIPs/MROPs, MROPs)** for query based systems and REST APIs, often inspired by relational algebra but used as reusable templates (filtering, ordering, subset, disjoint, etc.) [12,15,19,21,37,39].
 - **General MR pattern frameworks** and “family trees” of MRPs for MT, metamorphic exploration, and robustness [14,21].
 - LLM testing catalogues (e.g., 191 NLP MRs for LLMORPH) are also taxonomy style collections [48].
- **Search / ML / mining based MR discovery** operating over syntactic templates or feature spaces:
 - **Search based numeric MR inference** polynomial MRs [25], AutoMR with PSO+SVD [26], GEP based symbolic MR inference [27,28].
 - **Genetic programming** to generate executable MRs or assertion style MRs (GenMorph, GAssertMRs) [31,32].
 - **Composite MR search** via genetic algorithms and empirical study of composite depth [29,30].
 - **Execution mining** KABU's dynamic inference of “likely” MPs from executions [23]; MR Scout's mining of MR encoded test cases [38]; MetBench as a benchmark corpus of 924 numerical expression MRs [53].
 - **ML prediction of MR applicability** from program structure and documentation: graph kernel and RBF based MR predictors [46,47,50,52], text mining (MRpredT) [45], and lightweight code metric predictors [55].
 - **NLP/documentation based extraction**: MeMo infers equivalence MRs from Javadoc [40].
- **Model based (but non algebraic) MR generation**:
 - **Causal DAGs** define expected input–output causal structures from which MRs and tests are systematically generated [34].
 - **Object protocol abstraction (MemoRIA)** derives method equivalence MRs from protocol automata plus fuzzing and SAT based validation [33].
 - **Tabular expression specs** yield MRs from the structure of tabular requirements [16].
 -

Input pattern structural models prove any multi dimensional input pattern decomposes into basic one dimensional patterns for scientific programs, and provide an identification algorithm [17].

- **Abstraction/refinement** (MARS) iteratively refines domain agnostic relations into domain specific MRs using model checking and dynamic analysis [44].

Multiple surveys explicitly codify this taxonomy—expert/manual, pattern/taxonomy, search/ML, execution mining, data mutation, and NLP/LLM based methods—without introducing an operator algebraic category [2,3,4,5,6,7,9,54].

Role of Algebra, Invariants, and Symmetries in Existing MT Work

Algebraic terminology is descriptive, not a generative operator calculus

Several strands mention algebraic notions—associativity, commutativity, symmetry, monotonicity, conservation—but use them as *labels* or *inspiration* rather than as a structured operator algebra:

- **Surveys** catalog “algebraic” MR types (e.g., additive, multiplicative, periodic, subset) and contrast them with invariants and other relations, but do not define a formal algebra whose laws *generate* MR families [5,6].
- **Query based MRP**s explicitly “resort to relational algebra” to motivate patterns for filtering, projection, join, ordering, etc. [12,15,19]. However:
 - Relational algebra operators serve as a *vocabulary* to describe queries and patterns, not as a formally closed algebra with equivalence rules that are used to derive MR classes and boundaries.
 - There is no explicit algebraic analysis of where these patterns break (e.g., under non set semantics, stateful queries), nor any operator block coverage metric over relational operators.
- **Numeric and scientific MRs** routinely come from algebraic identities, monotonicity, conservation laws (probability sums to 1, sortedness preserved, etc.), especially in nuclear and scientific codes [18,56,54]. But:
 - These laws are cited as domain equations or “natural properties,” not organized into a program family operator algebra with stated composition rules and equivalence relations.
 - Boundaries (e.g., breakdown of numerical approximations) are discussed as domain caveats rather than as formal algebraic scope conditions.
- **Input pattern decomposition** provides algebraic flavored structure on the input side: any input pattern can be expressed as a combination of basic patterns in scientific computing [17]. Yet:
 - This concerns only the *input pattern space*, not the semantics of operators composing the program under test.
 - It does not derive MR classes as consequences of operator equivalences or monotone operators over the program family.

In short, algebraic properties and symmetries appear as **informal sources of MR ideas** or as part of MR taxonomies, but there is no prior systematic treatment that:

1. Defines a **set of primitive operators over a program family**.
1. States **composition/equivalence laws** over those operators.
1. Systematically derives **MR classes or MetaPatterns from those laws**, with explicit validity conditions and boundary characterization.

Treatment of MR Origin, Boundaries, and Coverage in Prior Work

MR origin/provenance is mostly informal or taxonomic

- **Manual/domain driven origin** dominates:
 - Surveys and case studies repeatedly emphasize that MRs rely heavily on **domain expertise** and understanding of the SUT [5,6,54].
 - Nuclear software work traces MRs to **physics models, numerical algorithms, or code level assumptions**, grouped into hierarchical layers (MRHM) [18,56]. This is explicit provenance but at a semantic layer level, not in terms of primitive operators and algebraic laws.
- **Taxonomy/pattern provenance**:
 - METRIC/METRIC+/CCML tie MRs to **input/output domain category choices** and MR patterns; origin is “selected pattern/category for this spec frame” [8,10,13].
 - MRP/MROP work for QBS, REST, and model transformations traces concrete MRs to **pattern catalogues**, MRIPs/MROPs, or mined generic trace patterns [12,14,15,19,37,39].
- **Search/ML provenance** is mostly algorithmic:
 - Search based and ML methods attribute MRs to **the search, optimization, or learning algorithm** (e.g., PSO, GP, GEP, graph kernel classifier) [23,25,26,27,29,30,31,32,33,35,36,38,40,45,46,47,50,52,55].

- Provenance is: “this MR was discovered by AutoMR/GenMorph/MeMo on this dataset,” not “this MR arises from these operator algebra laws.”
- **LLM based origins** semantic but opaque:
 - LLM methods (MR Adopt, SVPrompt MR, EMR generation, LLM capability studies) treat MR origin as coming from **LLM inference over prompts, documentation, code, or examples** [41,42,43,51].
 - The underlying structure is not formalized; provenance is the prompt and model rather than a structured algebra.

No prior work in this corpus attaches every MR or MR class to **named operator blocks and explicit algebraic laws** governing those blocks.

Boundaries and validity conditions are rarely formalized

- Most methods implicitly constrain scope by:
 - Limiting to **numeric programs** or certain data types [25,26,31,32,35,36].
 - Constraining to **equivalence** or **polynomial** MRs, or to pre defined MR types [25,26,40,46,47,52].
 - Focusing on specific **domains** (QBS, REST, CPS, nuclear) [12,15,19,32,56].
- Some works explicitly flag “likely” MRs:
 - KABU and Troya et al. call their inferred relations “likely metamorphic properties/relations” because they reflect observed executions and may encode implementation bugs [23,37].
 - Nuclear work distinguishes static (theory based) vs dynamic (data fitted) “likely” MRs and notes that the latter lack theoretical proof [18].
- **Model misspecification** is acknowledged but not algebraically framed:
 - Causal MT notes that MRs derived from misspecified DAGs can be unsound, and studies robustness to such misspecification [34].

Even in the most structured work, **boundaries are expressed as domain or model caveats**, not as precise **algebraic scope preconditions and out of scope strata** where particular operator laws cease to hold.

Coverage metrics are almost never operator level

Coverage in MT work is overwhelmingly defined at the **test or code level**:

- **Code coverage and mutation score** (fault detection effectiveness) are the standard metrics for MR sets [10,18,22,26,31,32,38,41,48].
- Some work counts the **number/diversity of MRs** or the proportion of MR types covered (e.g., how many patterns instantiated) [19,37,48].
- Fu et al. define **MT adequacy criteria** and a measurement framework, but still within the space of **test inputs, MR usage, and code coverage**, not operator coverage [7].

No prior paper defines coverage **over an operator algebra**, such as:

- Which primitive operators or combinations are exercised by a given MR set.
- Which equivalence/transform rules in an operator calculus have corresponding MRs.
- How completely the operator blocks of a program family are covered.

This is precisely the dimension that NOETHER introduces as “operator block coverage” [1].

Where NOETHER Fits and What It Contributes

NOETHER’s operator algebraic framework

NOETHER [1] is unique in this corpus in that it:

- Assumes an **upstream operator algebra** for a program family, decomposed into eight explicitly named blocks (symmetry, order, self adjoint, time reversal, limit, qualitative dynamics, method comparison, relational equivalence), each with documented scope preconditions.
- Defines a **constructive MetaPattern calculus**:
 - MetaPatterns are **equivalence classes over MRs**.
 - The CONSTRUCT MP algorithm mechanically derives a set of MetaPatterns from the operator algebra and proves:
 - **Closure** of MetaPatterns under a Translate operator (Theorem 1).
 - **Polynomial time decidability** of MetaPattern generation under a finite generating set (Theorem 2).
- Emphasizes **traceable algebraic origin**:
 -

Each MetaPattern and derived MR is linked to specific operator blocks and algebraic identities or composition rules; exact origin is part of the evidence [1].

- Makes **boundaries** a first class concept:
 - Explicitly defines **out of scope strata** (operator regimes and conditions where a block's laws do not hold).
 - Uses counterexamples to falsify an absolute completeness conjecture and to delineate *where* the current algebra fails, identifying extension directions for the operator system and Translate operator [1].
 - Provides falsifiable predictions (e.g. L* blindness for certain homogeneity preserving mutators) that are empirically checked in scope [1].
- Introduces **operator block coverage** as an evidence dimension:
 - Coverage is measured over the **space of operator blocks and their generated MetaPatterns**, not just over test inputs or code.
 - Per block complementarity is explicitly discussed—e.g., how symmetry vs order vs method comparison blocks yield qualitatively different MR families and thus cover different parts of the operator algebra [1].

Empirical work includes deriving concrete MRs (e.g., rotation equivariance for equivariant ML, Boltzmann reactor catalogue systematization) and comparing against a GP evolved baseline on a scope matched stratum where NOETHER's MetaPattern derived MRs dominate [1].

How this differs from and complements existing MR identification methods

Relative to the established paradigms, NOETHER:

- Is **not** a replacement for expert, taxonomy, search/ML, or LLM based MR identification; instead it provides a **new orthogonal axis**:
 - **Expert/taxonomy methods** (METRIC, MRPs, nuclear MR hierarchies) can be seen as *empirical catalogues* of useful MRs and patterns. NOETHER offers a way to **re ground such catalogues** in operator algebras, making explicit which operator laws generate which pattern families and where they fail.
 - **Search/ML/GP/GEP methods** optimize over **syntactic or template search spaces**; they are powerful at discovering effective MRs but have weak structural provenance. NOETHER can provide **algebraically justified search spaces and priors**, or be used to assess whether discovered MRs fall within expected operator algebraic classes or probe neglected operator blocks.
 - **Model based and causal approaches** structure MRs around **graphs or abstractions**; NOETHER structures them around **compositional operators**, giving a different, complementary notion of structure.
 - **LLM based methods** treat MR identification as **semantic text/code understanding**; their outputs could be constrained, checked, or categorised by an operator algebra, enabling **LLM+algebra hybrid pipelines** (e.g., LLM proposes candidate transformations; NOETHER checks if they correspond to known operator blocks and whether scope conditions hold).
- Introduces **new evidence dimensions** for MR sets:
 - Prior work overwhelmingly equates MR quality with **fault detection effectiveness** sometimes supplemented by code/mutant coverage and MR count.
 - NOETHER foregrounds:
 - **Algebraic provenance** (mapping each MR to operator laws).
 - **Boundary characterization** (documented conditions under which each MR class is sound/unsound).
 - **Operator block coverage** (which parts of the operator algebra are exercised by the MR set).
 - This allows MR methodologies to be argued as **complementary** rather than purely competitive on fault detection.

Given the surveyed literature, it is accurate and defensible to position NOETHER as:

- The **first operator algebraic MR identification framework** in the MT literature;
- A method whose main contribution is **systematic derivation of MR classes and boundaries from an explicit operator algebra over a program family**, with formal closure/decidability results and **operator block coverage and traceable algebraic origins** as primary evidence dimensions, clearly distinct from and complementary to fault detection focused approaches in existing MT research [1,2,3,4,5,6,7,12,13,14,15,17,18,19,23,24,25,26,27,28,29,30,31,32,33,34,37,38,39,40,41,44,45,46,47,50,51,52,53,54,55].

Categories

Comparative Landscape of MR Identification Paradigms

High level paradigms and where each paper sits

The table below synthesizes the *primary MR identification paradigm* and *degree of structure/automation* across the most relevant papers. It uses a coarse categorization that you can map to your NOETHER positioning:

- **Manual / Expert driven** MRs created by humans from domain knowledge/specs/code.
- **Taxonomy / Pattern-based**: libraries of MRs or MR patterns/templates reused or instantiated.
- **Search / ML-based**: MRs discovered via optimization, learning, or rule mining in a predefined hypothesis space.
- **Model-based (non algebraic)** derivation from higher level models (causal graphs, specs, abstractions), but not from an operator algebra over program families.
- **Operator algebraic** MRs or MR classes systematically derived from an explicit algebra of operators with composition laws, scope, and boundaries.

Work	Primary MR identification paradigm	Role of algebra / structure	Automation level	Notes relevant to NOETHER framing
NOETHER [1]	Operator algebraic derivation of MetaPatterns (equivalence classes over MRs)	Explicit two layer operator algebra; 8 operator blocks (symmetry, order, self adjoint, time reversal, limit, qualitative - dynamics, method comparison, relational equivalence); theorems on closure and decidability; explicit scope/boundary strata; operator block coverage	Automatic MetaPattern derivation from curated operator algebra; MR instantiation still requires connecting MetaPatterns to concrete systems	Only work in this set that explicitly turns an empirically curated operator algebra into a <i>generative</i> MR/MetaPattern calculus with explicit origin, closure, and documented out of scope boundaries [1].
MT/MR surveys [2,3,4,5,6,7,9,54]	Interpret/organize prior MR work as manual, taxonomy, search/ML, pattern, data mutation, etc.	Recognize algebraic properties (associativity, commutativity, etc.) as <i>categories</i> or examples, but do not provide an operator algebra that generates MR classes [5,6].	N/A	Provide the consensus classification that NOETHER can explicitly “plug into,” reinforcing that existing work is expert/taxonomy/search based rather than algebraic [2,3,4,5,6,7,9,54].
METRIC / METRIC+ / CCML/CCMT [8,10,13]	Category choice / taxonomy guided manual MR identification	Category choice framework over input (and in METRIC+ also output) domains; formal MR description language (CCML) and tool, but no operator algebra; categories are descriptive, not algebraically generated [8,10,13].	Tool supported manual MR derivation from specs/test frames; template like support	Canonical example of <i>taxonomy based</i> MR identification; no explicit derivation of MR classes from operator combinatorics, and no operator level coverage.
MR patterns for QBS / REST / generic MRPs [12,14,15,19,21,37,39]	Pattern/taxonomy based MRPs/MROPs as reusable templates	In QBS work, patterns are informally motivated by relational algebra, but relational algebra is a reference vocabulary , not a formal operator algebra with coverage/boundaries [12,15,19]. MRPs and MRIP/MROP pairs are abstract schemas; family trees of MRPs in [14] are taxonomic, not algebraic.	Mixed: often manual instantiation; [19,37,39] use patterns plus execution traces/specs to <i>automatically</i> instantiate many concrete MRs	Strong evidence that “algebraic” references in MT typically mean “inspired by relational algebra” rather than being a structured operator calculus with derivation/coverage; this is a key contrast point for NOETHER.
Domain hierarchical / physical equation driven MRs (nuclear, CRAM) [18,56]	Manual MR identification guided by hierarchical MR classification (physics, algorithm, code) and domain equations	Use physics equations and numerical algorithms as the source of MRs; MRHM hierarchy classifies MRs into levels [18,56]. No explicit operator algebra, no formal algebraic closure or coverage.	Manual, with classification and some static vs dynamic identification distinction [18]	Shows explicit <i>provenance</i> (physics vs algorithm vs code), but still not presented as an algebra of program operators; boundaries are mostly informal (where physical laws/approximations hold). Useful comparator for provenance vs NOETHER’s operator block origin.
Tabular expression based MR construction [16]	Structured manual derivation from specification tables	Uses structural relationships between header/main grids in tabular expressions to systematize MR construction. No operator calculus over program families.	Semi systematic but essentially manual	A domain independent structural method, but not an operator algebra; no operator block coverage notion.
		Proves that multi dimensional input patterns can be decomposed into basic one dimensional patterns for scientific com-	Algorithmic analysis of input patterns; MR identifica-	One of the few works with structural/decomposition theorems, but they are about <i>input pattern space</i>

Input pattern structural model & identification algorithm [17]	Structural modeling of input patterns	puting; provides structural model and algorithm [17]. Focused on input patterns only; no full operator algebra or MR class derivation.	tion still requires mapping patterns to semantics	rather than an algebra over program operators.
Causal graph based MR generation [34]	Model based from causal DAGs	Uses causal DAGs as behavioral models to derive MRs; MR classes correspond to causal expectations (e.g., functional dependence monotonicity) [34]. No operator algebra over program family operators; coverage is in terms of causal-relationship exercise, not operator coverage.	Automatic MR generation given a causal graph; still requires model specification or causal discovery	Good example of non algebraic but structured model based MR generation; operator algebraic NOETHER can be positioned as structurally analogous but distinct in its <i>operator</i> basis and explicit algebraic closure.
MARS: Model assisted refinement for scientific software [44]	Iterative abstraction refinement with dynamic analysis and human input	Starts from domain agnostic relations and dynamically discovered patterns; uses model checking to suggest refinements, then humans refine to domain specific MRs [44]. No operator algebra.	Semi automatic MR refinement loop	Emphasizes coverage of behaviors via abstraction refinement, but at the level of relations over I/O, not at the operator-block level.
Benchmarks / adequacy / catalogues (MetBench, MT adequacy, LLM MR catalogues) [7,48,53]	Empirical characterization, adequacy criteria, and MR datasets	MetBench models numerical expression MRs and encodes them for testing [53]; [7] proposes adequacy metrics for MT; [48] builds a large MR catalogue for LLM/NLP. None derive MR classes from program family operator algebras.	N/A	These works are important for empirical grounding and evaluation dimensions (fault detection, adequacy), but orthogonal to operator algebraic origin or operator coverage.

Comparison of Taxonomy/Pattern Based vs Search/ML Based MR Identification

Taxonomy and pattern frameworks vs operator algebraic derivation

The following table contrasts **taxonomy/pattern-based MR frameworks** with **search/ML-based automatic MR inference** regarding MR origin, generative basis, boundary treatment, and coverage notions.

Category & representative works	MR origin & generative basis	Boundaries & validity conditions	Coverage notions	Position vs NOETHER
METRIC / METRIC+ / CCML (category choice family) [8,10,13]	Human testers select MR categories based on input (and output) domain properties; METRIC+ adds output domain information [10]. CCML language formalizes MR descriptions; CCMT automates some steps [13].	Validity largely argued informally from specs/domain; no systematic explicit boundary characterization beyond category definitions.	Mostly test/data coverage and MR fault detection in experiments [10,13]; no coverage metric over <i>operator combinations</i> or category lattice closure.	Canonical taxonomy based MR identification; NOETHER can be contrasted as using an empirically curated operator-algebra decomposition as the generative basis, with explicit algebraic closure and block coverage [1].
MRPs/MROPs for QBS & REST [12,14,15,19,21,37,39]	MRs abstracted into patterns : MRPs, MRIPs/MROPs. Many QBS patterns explicitly “re-sort to relational algebra” for intuition [12] and MROPs define generic output relation types [15]. [19,37,39] automatically instantiate MRs from patterns plus traces/specs.	Boundaries typically stated at a high level (e.g., pattern applies when query filters are monotonic, when output types are sets/lists). No algebraic description of where relational algebra inspired patterns break (e.g., under non set semantics, side effects).	Coverage is primarily: number/diversity of instantiated MRs and their fault detection (e.g., mutation score in [37,19]); no explicit coverage over query - operator algebra or pattern space closure.	These works are closest in vocabulary to an “algebraic” view, especially via relational algebra [12,15], but still use it descriptively. NOETHER’s contribution is to formalize an operator algebra with closure properties and boundaries, and to treat coverage over operator blocks as a first-class metric [1].
	MRs derived from physics equations, numerical methods, and implementation details and classi-	Boundaries correspond to validity of physical models and numerical approximations; some explicit statements that “likely MRs” discovered dynamically need	Empirical coverage in terms of test effectiveness and layers exercised (unit, integration, system) [18];	Provides a clear notion of <i>MR provenance</i> , but at a semantic layer hierarchy not at a combinatoric operator algebra level. NOETHER adds a finer - grained operator block ori-

Domain hierarchical models (nuclear/CRAM) [18,56]	fied into physics / algorithm / code layers [18,56].	further theoretical justification [18].	no operator algebra coverage metric.	gin and boundary formalization [1].
--	--	---	--------------------------------------	-------------------------------------

Search/ML-based automatic MR identification

Work	MR hypothesis space & generative mechanism	Treatment of MR origin & boundaries	Coverage / evaluation	Relation to operator algebraic perspective
Polynomial MR inference [25]	Search based inference of polynomial MRs (equality/inequality relations) via metaheuristic search over polynomial templates; applied to numerical programs.	Origins: pre chosen polynomial templates and variable combinations. Boundaries: type/domain restrictions implicit; no explicit formal boundary characterization beyond template applicability.	Mutant kill rate and fault detection vs baselines; no operator level coverage.	Exemplifies template and search MR generation; hypothesis space is syntactic (polynomials), not defined as closure of an operator algebra.
AutoMR [26]	Particle swarm optimization over a space of equality/inequality relations; uses SVD and constraints for “cleansing” redundant MRs.	MR origin: PSO search in numeric relational templates. Boundaries: limited to numeric inputs/outputs; classification as equality/inequality; no explicit validity proof.	Fault detection via mutation and differential testing; redundancy reduction metrics.	Still purely search based and template driven; no operator algebra or MR-class derivation.
GenMorph & GAssertMRs [31,32]	Genetic programming over expression trees representing candidate MRs; domain specific input patterns (MRIPs) plus generic output constraints [32]; GenMorph uses fitness functions penalizing FPs/FNs [31].	Origins: GP search guided by pre specified primitives/templates. Boundaries: numeric/boolean/sequence domains and CPS QoS metrics; no explicit algebraic limits.	Evaluated by mutation score, number of effective MRs, and improvement over baseline tools [31,32].	The search space is a syntax of assertions or small transformations, not an operator algebra over program families.
KABU (dynamic inference of metamorphic properties) [23]	Records program executions and applies pre-defined input transformers (based on prior MR literature) to mine “metamorphic properties.”	Origins: combination of preselected input transformers and Daikon like invariant mining; MPs are “likely” and require human vetting. Boundaries: no explicit formalization; MPs reflect observed executions, possibly including bugs.	Precision vs student derived MPs; regression use across versions.	Here again, MRs are learned from executions; transformers are loosely inspired by MR catalogues, without an algebraic derivation.
GEP based MR inference (two papers) [27,28]	Gene Expression Programming to mine symbolic expressions that relate outputs, given preset input patterns.	MR origin: GEP search; boundaries: implicitly defined by preset input patterns and expression language.	Fault detection and comparison to static analysis MRs.	Another instance of search within a preset hypothesis class; not algebraically grounded.
GA based composite MR construction & layers study [29,30]	Genetic algorithms search over compositions of basic MRs (represented as sequences/combined operations) [29]; [30] explores how many composite layers are cost effective.	Origin: manual basic MRs, then automatic composition by GA; no explicit equivalence laws or algebraic structure beyond “composition” operator. Boundaries: computational (combinatorial explosion) and empirical (fault detection effectiveness peaks at ~3 layers) [30]; not semantic boundaries.	Evaluation: number of composite MRs and mutation score vs basic MRs [29]; cost effectiveness vs layers [30].	These works touch compositionality, but treat MR composition as a combinatorial search, not as an algebra with known laws and soundness boundaries .
µMT (data mutation directed) [20,22]	Provides data mutation operators and template style mapping rules to guide testers toward MRs [22]; earlier workshop version [20].	Origins: human uses mutation operators/templates; boundaries implicit in mutation operator semantics.	Evaluated on MR identification effectiveness and fault detection [22].	Essentially a hybrid of pattern guided and manual MR identification; the “operators” here are data mutations, not an algebra with formal equivalence/closure properties.
MR Scout & MR Adopt [38,41]	MR Scout mines MR encoded test cases and synthesizes parameterized “codified MRs” [38]; MR Adopt uses LLMs plus data flow refinement to infer general input transformations from a single encoded MR [41].	Origins: existing test suites (MR Scout) and encoded examples (MR Adopt); boundaries: generalization limits of mined/LLM synthesized transformations, validated empirically (e.g. proportion of successful generalization) [38,41].	Coverage measured in terms of test suites augmented, line/mutation coverage gains [38,41].	Learned directly from human artifacts and bounded by LLM/search capabilities; no operator algebra or coverage over operator combinators.
	Machine learning classifiers predict which from a fixed	MR origin: predefined MR set determined by experts;	Evaluated by classification metrics (accuracy,	These works treat MRs as labels; they do not attempt

Predicting / classifying MRs with ML (PMR line) [45,46,47,49,50,52,55]	MR list apply to a given method, using CFG based or source metric features [46,47,49,50,52,55].	classifiers predict applicability. Boundaries: MR vocabulary and training corpus; no derivation of new MR classes.	AUC ROC) and, sometimes, impact on testing [46,47,55].	to derive MR families or an underlying algebra.
MeMo (from Javadoc) [40]	NLP to extract equivalence MRs from natural language comments.	Origins: documentation; boundaries: limited to equivalence type MRs captured in prose; no explicit boundary proofs.	Evaluated on MR extraction precision/recall and fault detection [40].	Adds documentation as another MR source; still no operator algebra.
Abstraction aware MR inference (MemoRIA) [33]	Builds object protocol abstraction, uses fuzzing to propose candidate method equivalence MRs, and validates/deduplicates them via SAT based analysis.	Origins: protocol abstraction plus fuzzed execution; boundaries are constrained by abstraction fidelity and fuzzing coverage; no formal algebraic boundaries.	Evaluated against competing MR inference (SBES) on number/quality of inferred equivalence MRs [33].	Uses <i>behavioral abstraction</i> as structural basis, but again not an operator algebra over a program family.
Ocean model MR search (Hiremath et al.) [35,36]	Models input transformation g as (initially) affine maps and searches for g such that some evaluation e is preserved; cost penalizes trivial g and encourages orthogonal transformations [35,36].	Origins: search over transformation parameters; physical symmetries are post hoc interpretations; boundaries: affine assumption, cost definition.	Evaluated on two ocean - model implementations; complexity and interpretability issues discussed [35,36].	Approaches domain symmetries but does not formalize a symmetry operator algebra; transformations are numeric parameterized and discovered via optimization.
ARCAMETES, MRpredT, etc. [24,45]	ARCAMETES learns MR patterns via association rule learning and combinatorial testing [24]; MRpredT uses text classification to predict MR types from documentation [45].	Origins: mined patterns or trained classifiers; boundaries implicit in learning algorithm and training data.	Evaluated empirically (prediction performance, MR utility) [24,45].	Yet more evidence of <i>search/ML centric</i> MR identification; no operator level algebra.

LLM Based MR Identification vs Operator Algebraic NOETHER

LLM centric MR discovery and its evidence dimensions

Work	LLM's role in MR identification	Evidence dimensions emphasized	Relation to NOETHER's origin/boundary/coverage dimensions
LLMs for MR discovery (survey) [9]	Catalogues numerous LLM assisted MR identification tools: translating legal texts into metamorphic specs, MR Scout+LLM variants, MR Coupler (method coupling + LLM inference), MR Adopt, etc.	Summarizes empirical performance, MR usefulness, application to code and behavioral testing [9].	Positions LLMs as powerful <i>knowledge mining</i> engines, but MRs are drawn from human artifacts (docs, code, tests); origin is textual/structural, not an operator algebra.
MR Adopt [41]	Infers input transformation functions for encoded MRs using LLM synthesis plus refinement/validation loop.	Evidence: % of transformations that generalize to all prepared test cases (72%), coverage and mutation score improvements [41].	Origin tied to encoded MR examples, not operator combinators; boundaries are empirical generalization limits, not algebraic validity conditions.
SVPrompt MR [42]	Proposes self verification prompting for MR identification using deepseek R1 on a nuclear thermal hydraulic code.	Evidence: coverage rate, accuracy rate, and "innovation rate" of LLM identified MRs relative to expert MRs; LLM covers all 44 expert MRs and finds 24 new ones [42].	Provides quantitative <i>MR coverage vs experts</i> , but coverage is defined over the set of manually known MRs, not over an operator algebra.
Generating executable MRs with LLMs [43]	Few shot prompting to derive executable MRs (EMRs) from requirements/API specs.	Evidence: developer questionnaire and empirical accuracy of EMRs for a web app [43].	Again, origin in textual requirements, and evaluation via usefulness/accuracy; no concept of operator blocks or algebraic provenance.
Can LLMs discover MRs? (SANER study) [51]	Large scale empirical study of GPT 3.5/others on MR identification across 37 SUTs, using prompts derived from prior MT studies.	Evidence: systematic evaluation of MR correctness/quality vs existing MR sets; explores LLMs' capabilities and limitations [51].	Frames LLMs as alternative MR identification engines, but with no underlying algebraic structure; MR origin is still primarily "LLM from prompt + examples."

Contrast to NOETHER: LLM based approaches treat MR identification as *asemantic inference* from natural language, code, and examples; NOETHER treats it as a **constructive derivation** from an explicit operator algebra, with:

- traceable algebraic origin (which block/law generated a MetaPattern) [1];
- explicit scope/boundary strata for when an operator law is valid or not [1];

- operator block coverage as an evidence dimension distinct from fault detection [1].

No LLM based work in this corpus defines or measures operator level coverage or provides algebraic proofs of MR class closure.

Comparative Treatment of MR Origin, Boundaries, and Coverage

MR origin/provenance

- **Explicit provenance as “domain layer”**
 - Nuclear and CRAM MRHM works [18,56] explicitly separate **physics model MRs**, **algorithm MRs**, and **code MRs**, and distinguish static vs dynamic MR discovery [18].
 - MetBench [53] documents MR sources (open source, academic, industrial software).
- **Pattern level provenance**
 - MRP/MROP work [12,14,15,19,21,37,39] traces MRs to general query/REST semantics or prior pattern catalogues.
 - METRIC/METRIC+ / CCML [8,10,13] trace MRs to input/output domain characteristics and category choice decisions.
- **Search/ML provenance**
 - Search based and ML approaches attribute MRs to their underlying search or training process (e.g., PSO search, GP, GEP, causal graph models) [23,25,26,27,29,30,31,32,33,35,36,38,40,45,46,47,49,50,52,55].
 - Provenance is *algorithmic* (“found by AutoMR/GenMorph”) but not semantic; they do not usually maintain a decomposed trail back to a primitive operator law.
- **NOETHER’s operator algebraic provenance**
 - NOETHER defines an **empirically curated operator algebra decomposition** into eight blocks and derives MetaPatterns algorithmically, tying each to its generating operator block(s) [1].
 - This makes provenance *structural*: each MetaPattern and MR instance can be linked to specific algebraic laws and operator block compositions [1].

Boundaries / validity conditions

- Many works use **informal or local boundaries**:
 - Limited to **numeric input/output** or specific template families [25,26,31,32,35,36].
 - Applicability to certain **domains** (query systems, model transformations, scientific computing) [12,15,19,37].
 - “Likely” MRs inferred from data are explicitly warned as not universally valid [23,37].
- Nuclear software papers [18,56] make **physics and algorithm limits** explicit (e.g., model assumptions, numerical schemes), but still do not phrase them as algebraic boundaries over operators.
- Causal graph work [34] discusses misspecification of causal models and its impact on MR soundness, again as model validity boundaries, not algebraic ones.
- NOETHER uniquely:
 - **Defines out of scope strat** for its operator algebra (where particular algebraic laws do not hold, e.g., floating point non associativity, broken symmetries) [1].
 - Demonstrates **falsifiable predictions** about where its operator algebraic patterns will fail (e.g., L* blindness on homogeneity preserving mutators), and documents counterexamples that falsify an “absolute completeness” conjecture (APWR) [1].
 - Treats boundary characterization as a central, **falsifiable** component of the contribution, not just incidental domain caveats.

Coverage definitions

- **Standard MT coverage metrics**
 - Most works evaluate coverage in terms of **code coverage** and **mutation (fault) coverage**, sometimes line/branch coverage [10,18,22,26,31,32,38,41,48].
 - Some address **MR coverage** merely as the number or diversity of MRs instantiated [19,37,53].
- **Test adequacy criteria for MT**
 - Fu et al. [7] focus specifically on **test adequacy for MT**, rigorously defining adequacy criteria and measurement for MT regimes. Their metrics consider MR identification and source inputs as key elements but still operate at the test/MR level, not at an operator block algebra level.
- **Behavioral/abstraction coverage**
 - MARS [44] uses behavioral coverage under abstraction/refinement; causal MT [34] speaks of exercising structural causal relations; MemoRIA [33] aims to cover more behaviors via object protocol abstraction.

- **NOETHER's operator block coverage**
 - NOETHER introduces **operator level derivability and coverage** MetaPatterns are constructed to cover the blocks of the operator algebra, and the framework explicitly treats **per block complementarity and coverage** as an evidence dimension [1].
 - This is distinct from test or mutant coverage: coverage is measured against the space of operator combinations and algebraic laws, not just code or input space.

Summary: Where NOETHER Sits Relative to the Field

Across the corpus:

- **Consensus taxonomy** from surveys [2,3,4,5,6,7,9,54] supports a three way structural division: manual/expert, taxonomy/pattern based, and search/ML based MR identification (with some model based subcases such as causal graphs [34] and protocol abstractions [33]).
- **No other work** in this set systematically:
 - defines a **program family operator algebra** with primitive operators, composition/equivalence laws,
 - derives **classes of MRs (or MetaPatterns)** from that algebra with explicit closure properties,
 - and treats **operator block coverage and algebraic origin/boundaries** as first class evidence dimensions.

The closest precursors are the relational algebra inspired MRPs/MROPs for query systems [12,15,19], structural input pattern decompositions [17], MR hierarchies over physics/algorithm/code [18,56], and abstraction based methods (causal graphs [34], protocol automata [33], model refinement [44]). Yet all of these:

- use their structures either as **inspiration, classification schemes, or model based templates**
- and do **not** present a constructive operator calculus that generates MR classes and makes block coverage and algebraic boundaries explicit.

This gap is precisely where NOETHER [1] can be framed as complementary:

- It does not primarily compete on fault detection effectiveness (though it evaluates against a GP baseline);
- Instead, it contributes:
 - an operator algebraic generative framework for MR/MetaPattern classes,
 - explicit, falsifiable boundary characterizations, and
 - operator block coverage and traceable algebraic origin as **orthogonal evidence dimensions** to traditional fault detection metrics.

Timeline

Chronological Phases in Metamorphic Relation Identification

Phase 0 (pre 2013): MT concept established, MR identification largely manual

This phase is mostly summarized retrospectively by later surveys. MRs were:

- **Hand crafted by experts** from domain knowledge or specifications.
- Often inspired by **algebraic laws, invariants, symmetries, and monotonicity**, but without an explicit calculus that generates MR classes.

Segura et al. [5] and Chen et al. [6] document this period as dominated by **manual MR derivation**, with a few domain specific catalogues (e.g., numeric computations, search, ML).

Phase 1 (2013–2016): First automation attempts and foundational surveys

Key developments

1. **ML based MR prediction (program analysis features)**
 - Kanewala & Bieman's line of work [46,47,52] introduces **predictive models** (graph kernels, code metrics) to decide which predefined MRs apply to a given program.
 - Programs are represented via **control flow graphs** features feed supervised ML classifiers that output MR labels.
 - Conceptually: MR identification shifts from **purely expert driven** to **ML assisted classification over a fixed MR vocabulary**.

1. Search based inference for numeric programs

- Zhang et al. (polynomial MRs) [25] use **search/optimization** to fit polynomial equalities/inequalities to execution data.
- This inaugurates the “**MR discovery as optimization**” line: MR is a syntactic expression, search optimizes coefficients/structure s.t. observed executions satisfy it.

1. Field level consolidation: the 2016 MT survey

- Segura et al. [5] provide the first broad survey emphasizing that MT workflows have a distinct **MR construction step**, and catalog MR types (algebraic, ordering, inclusion, periodicity, convergence, etc.).
- They explicitly note that MR construction is **manual, domain driven, and a major bottleneck**[5].
- No operator algebra is proposed; algebraic examples are illustrative, not generative.

1. METRIC and early structured MR identification

- METRIC (METamorphic Relation Identification based on the Category choice framework) [8] appears as an early **systematic MR identification protocol**, though the abstract is missing here.
- Later work describes METRIC as a **category choice, pattern based method** that guides experts through structured choices over input categories [8,10].

1. KABU: dynamic inference of “likely” metamorphic properties

- Su et al. [23] infer “metamorphic properties” dynamically by instrumenting code, applying fixed input transformers, and mining relations between arguments and outputs.
- This is an early **data driven MR mining** approach, but with **transformers manually designed** from existing MT practice; there is no explicit operator algebra.

Conceptual state at the end of Phase 1

- MR identification is recognized as **central and difficult**.
- Two main automation strands emerge:
 - **Prediction**: classify code into predefined MR labels (Kanewala et al. [46,47,52]).
 - **Search/mining**: infer numeric MRs from traces (Zhang et al. [25], Su et al. [23]).
- Theory is consolidated by [5]; “algebraic” terminology is used descriptively, not as a formal operator calculus.

Phase 2 (2017–2019): Patterns, taxonomies, and broader automatic MR generation

Patterns and taxonomies as reusable MR abstractions

1. Metamorphic Relation Patterns (MRPs)

- Troya et al. (model transformations) [37,39]:
 - Define MRIPs (input patterns) and MROPs (output patterns), and a **catalogue of 24 domain independent MRs** plus five generic trace patterns used to infer likely MRs [37].
 - MRs are “likely” because universal validity is not proven.
 - This is a **pattern/taxonomy plus trace mining** approach.
- Segura et al. (query based systems, QBS) [12]:
 - Introduce **MR patterns defined against relational algebra** (filtering, ordering, etc.).
 - Relational algebra operators inspire MRPs, but the paper does **not** build a closed operator algebra or derive MR classes via algebraic laws; patterns are templates.
- Segura et al. (RESTful Web APIs) [15]:
 - Define **Metamorphic Relation Output Patterns (MROPs)** (Equivalence, Equality, Subset, Disjoint, Complete, Difference) and instantiate them into concrete MRs.
 - This is a canonical **domain taxonomy of MR output patterns**.

1. MR pattern framework for exploration/robustness

- Zhou et al. [21] and later Ying et al. [14] elevate MRPs to a general abstraction:
 - Symmetry and “change direction” patterns [21];
 - Formal MRP family trees and a classification framework [14].
- These works deepen **pattern level reasoning** but patterns are still **manually curated** rather than algebraically generated.

Broader automatic MR generation

1. AutoMR: search based inference and cleansing for numerous MRs

-

Zhang et al. [26] extend polynomial search [25] with PSO and SVD/constraint solving to infer and “cleanse” equality/in-equality MRs for numerical programs.

- This is archetypal **search based MR discovery** with improved scalability and de duplication, still confined to numerical expression templates.

1. Domain specific and template oriented methods

- **MT (data mutation)** [20, later extended in [22]]:
 - Provides **data mutation operators** and **template style mapping rules** to guide testers in MR identification.
 - Automation is **operator template + human** not operator algebra: operators are mutation operators on data, not a semantic operator algebra over program families.
- **Tabular expressions** [16]:
 - Derive MRs from tabular expression specifications by exploiting structural relationships between header grids and main grids.
 - This treats the table as a structured spec, but not as a compositional operator algebra with equivalence laws.
- **Nuclear domain MR hierarchies** [56]:
 - Li et al. construct a three layer MR hierarchy (physics, algorithm, code) for nuclear software and derive MRs from CRAM and burnup equations [56].
 - This strongly emphasizes **domain theory provenance** but still at the level of categorized relations, not a formal operator calculus.

1. Second major MT survey

- Chen et al., “Metamorphic Testing” [6], ACM CSUR 2018, solidify MT theory:
 - Formal MT model, broad review of applications, and discussion of MR identification as an open challenge.
 - They acknowledge pattern/catalogue approaches and early automation, but **no operator algebraic MR derivation** appears.

Phase 2 summary

- The field converges on **patterns/taxonomies** as a dominant way to organize MRs (MRPs, MRIPs/MROPs, MROPs for REST, MR family trees).
- Automation expands via **search, mutation, trace mining** mostly confined to numeric or domain specific templates.
- Algebraic structures (relational algebra, tabular expressions) are influential inspirations, but not developed into **operator algebras that generate MR classes and boundaries**.

Phase 3 (2020–2022): Industrialization, composite relations, and new specification driven generators

Industrial/real world focus and composite relations

1. Composite MR construction by search

- Xiang et al. [29] view composite MRs as sequences of simpler MRs and use a genetic algorithm to search for high quality composites.
- Zhang et al. [30] study **number of composite layers vs. cost/effectiveness**, finding three layers most cost effective.
- This deepens the **search based composition** line but still over a predefined MR primitive set; no operator calculus is defined.

1. Industrial case studies and domain abstraction

- Ayerdi et al. (CPS, GAssertMRs) [32]:
 - Use genetic programming over **domain specific MR input patterns** and a generic output template, grounded in a database of correct/incorrect CPS executions.
 - Strong industrial evaluation, but the generative space is a **template space**, not an operator algebra.
- Nuclear software verification [18]:
 - Propose an MR hierarchy (physics, computational, code, likely MRs) and emphasize **static (equation based) vs dynamic (fitted) MR derivation** [18].
 - Highlight MR provenance and relative rigor, but not as algebraic operator derivation.

1. Specification driven MR construction

- METRIC+ [10,11]:
 - Extends METRIC [8] by using both input and output domain “category choice” information to guide MR identification from complete test frames.

- This formalizes MR identification as a **guided combinatorial reasoning** over spec frames; still taxonomy /category driven.
- **Tabular expression MR construction**[16] and **input pattern structural model** [17]:
 - Yan et al. [17] introduce the notion of **basic input patterns** and prove that any input pattern can be expressed as a combination of these in scientific computing programs.
 - They present a **structural model of input patterns** and an identification algorithm with complexity analysis.
 - This is one of the closest precursors to an “algebraic” perspective on input side transformations, but:
 - Focuses on **input patterns only**, not a program family operator algebra.
 - Does not derive **MR classes with explicit semantic boundaries** or operator coverage metrics.

1. Automated MR generation for QBSs from parameter specs

- Segura et al. [19] automate MR generation for QBSs using a lightweight parameter specification and MR patterns [12,15].
- Hundreds of MRs are generated in seconds for systems like IMDb and YouTube.
- This is a **pattern instantiation engine** over a domain taxonomy, not an operator algebra.

1. Systematic literature review on MR automation

- Altamimi et al. [4] (SLR, 2022) map MR automation techniques:
 - Distinguish methods that analyze program structure (CFG/PDG, initial MRs) vs execution based methods using random/specific inputs.
 - Conclude the field is **still in its infancy** and call for more ML based methods.
 - They still categorize approaches as **manual, pattern based, search based, ML based** there is no operator algebraic axis.

Phase 3 summary

- There is a strong push toward **industrial applications**, composite MR reasoning, and spec guided generators (METRIC+, QBS MR generators, nuclear hierarchies, CPS case studies).
- Structural models (basic input patterns [17]) appear, hinting at algebraic decompositions, but still stay at the level of **pattern combinatorics**, not an explicit operator algebra over program semantics.

Phase 4 (2020–2024): Search/GP, NLP, documentation mining, and adequacy theory

Automated MR synthesis from diverse sources

1. Genetic programming and stochastic search

- **GenMorph** [31] and **GAssertMRs** [32] generalize search based MR generation:
 - Use genetic programming to search executable MRs (GenMorph) or MR assertions (GAssertMRs), optimized for low false positives/negatives and compactness.
 - Input spaces are boolean/numeric/sequences [31] or CPS specific QoS metrics [32].
 - These are powerful **black box search** over MR expression templates, not operator structural methods.
- **GEP based MR identification**[27,28]:
 - Transform MR recognition into a **symbolic expression regression** problem with gene expression programming, based on pre set input patterns and mined output patterns.
 - Again, template+search centered on expression syntax.

1. Mining MRs from existing artifacts

- **MeMo** [40]:
 - Automatically derives **equivalence MRs** from natural language Javadoc.
 - Treats MR types as predefined and uses NLP to extract them; thus, **documentation based MR identification**
- **MR Scout**[38]:
 - Mines “MR encoded” developer test cases and synthesizes them into parameterized “codified MRs,” achieving high coverage and mutation scores.
 - Emphasizes **mining human practice** from tests.
- **MRpredT** [45] and **Duque Torres et al.**[55]:
 - Text mining and code metric ML models to predict applicable MRs from documentation and source metrics respectively.
 - These refine the **predictive model** line started in [46,47,52].

1. Causal and model assisted generation

- **Causal graph based MT**[34]:
 - Uses **causal DAGs** as specifications and systematically generates MRs as consequences of the causal structure, with automated test generation.
 - This is a clear example of **model based MR generation** but:
 - The model is **causal, not algebraic**.
 - There is no operator calculus, no MR class boundaries in algebraic terms.
- **Model Assisted Refinements (MARS)**[44]:
 - Vision paper for iteratively refining domain agnostic relations into domain specific MRs using model checking and dynamic analysis.
 - Focuses on **abstraction/refinement loop**, not on operator algebra.

1. Adequacy and benchmarks

- Fu et al. [7] propose **MT adequacy criteria**, explicitly surveying MR identification methods (pattern/template, ML, category choice, search, data mutation, NLP).
- MetBench [53] builds a **benchmark dataset of 924 numerical expression MRs** to support MR research and tool evaluation.

1. Comprehensive MR generation surveys

- Rui Li et al. [2,3] (2024) provide a **state of the art review of MR generation**
 - Classify methods into pattern/taxonomy, search /ML based, execution mining, etc.
 - Highlight that current techniques are often domain limited, template restricted, and of varying MR quality.
 - There is no mention of **operator algebraic derivation of MR classes** instead they call for more systematic and general MR generation frameworks.

Phase 4 summary

- Automation becomes **diverse and multi source** GP, GEP, ML over code/doc, mining from tests, causal models.
- Surveys [2,3,7] consolidate the classification of MR identification into **expert/manual vs pattern/taxonomy vs search/ML/execution mining** without an operator algebra category.
- “Algebraic” remains an **informal label** (e.g., polynomial MR templates, relational algebra inspired MRPs) but not a generative calculus with explicit boundaries and coverage.

Phase 5 (2024–2026): LLM based MR identification and the emergence of NOETHER

LLMs as a new automation layer

1. LLM driven MR and EMR synthesis

- **MR Adopt**[41]:
 - Deduces **input transformation functions** from a single encoded MR using an LLM plus data flow based refinement and validation.
 - Focuses on recovering reusable transformations from hard coded examples.
- **LLM based executable MR generation**[43]:
 - Uses few shot prompting to generate **executable metamorphic relations (EMRs)** from requirements/API specs.
- **SVPrompt MR**[42]:
 - Proposes a **self verification prompting strategy** for MR identification, evaluated on a nuclear thermal hydraulics code.
 - Compares against existing methods and reports higher MR coverage and accuracy.
- **LLM capability study** [51]:
 - Large scale empirical evaluation of LLMs (gpt 3.5 etc.) on MR identification across 37 SUTs from prior MT studies, framing LLMs as a new, partially automated source of MR suggestions.
- **Bidirectional MT–LLM survey** [9]:
 - Catalogue of 93 studies on **MT for LLMs** and **LLMs for MT**, documenting numerous LLM assisted MR discovery approaches (from legal texts, code, tests, single I/O pairs, etc.).

1. Integrated MT systems and languages

- **CCML/CCMT [13]:**
 - Introduce a **general metamorphic specification language** and an integrated tool that supports MR description, automatic test input generation, and composite MRS.
 - Builds atop the category choice framework (METRIC, METRIC+) and MR composition work; still **apattern /category oriented MR language** not an operator algebra.

1. Domain catalogues for LLM testing

- Cho et al. [48] compile a **catalogue of 191 NLP MRs** and implement 36 in LLMORPH for MT of LLMs on NLP tasks.
 - The catalogue is literature driven and manual/taxonomy style [48].

NOETHER (2026): operator algebraic MR derivation

1. NOETHER framework [1]

- Introduces a **two layer operator algebraic framework for metamorphic pattern (MetaPattern) discovery**
 - Upstream: assumes a **curated operator algebra decomposition** into eight blocks (symmetry, order, self adjoint, time reversal, limit, qualitative dynamics, method comparison, relational equivalence), each with explicit scope pre-conditions.
 - Downstream: CONSTRUCT MP **mechanically produces a MetaPattern set**, with:
 - Algebraic closure under a Translate operator (Theorem 1).
 - Polynomial time decidability under a finite generating set (Theorem 2).
 - MetaPatterns are formalized as **equivalence classes over MRs** with **traceable algebraic origin** and **per block complementarity**.
- The framework explicitly emphasizes:
 - **Origin:** every MetaPattern is justified by specific operator identities and composition rules in the operator algebra.
 - **Boundaries:** explicit **out of scope strata** where the algebra's laws (and thus the induced MR classes) do not hold.
 - **Operator level coverage** which operator blocks and equivalence rules are exercised by the instantiated MRs.
- Examples:
 - Deriving **rotation equivariance MRs** for equivariant ML from symmetry block operators.
 - Systematizing a catalogue for a **Boltzmann reactor** via method comparison and qualitative dynamics blocks.
- Empirical results include:
 - Head to head comparison vs a GP baseline, with NOETHER's MetaPattern set dominating on a scope matched stratum.
 - A **falsifiable prediction** of L* blindness on certain mutators that holds in scope.
 - Documented **counter examples** that falsify an absolute completeness conjecture and identify extensions to the Translate operator and operator blocks.

Phase 5 summary

- LLM based MR identification **adds a new automation channel**, still largely operating in the space of **pattern descriptions or code templates** inferred from natural language or examples.
- NOETHER is the **first explicitly operator algebraic MR identification framework** in this corpus:
 - Defines a **program family operator algebra** (blocks + laws).
 - Derives **MR classes (MetaPatterns)** via algebraic reasoning, with explicit **scope, boundaries, and coverage notions**.
 - Treats **operator block coverage and algebraic provenance** as primary evidence dimensions, distinct from fault detection.

Emerging Method Families and Shifts in Emphasis

From expert driven to pattern/taxonomy reuse

- Early MT practice (before ~2013) and the 2016 survey [5] characterize MR identification as **expert intensive and manual**
- Phase 2 brings **MRPs, MRIPs/MROPs, MROPs for REST, MR family trees** [12,14,15,21,37]:
 - MRs are grouped into patterns that can be reused across systems.
 - Identification becomes: choose patterns + instantiate parameters.
- These taxonomies are **descriptive, not generative**: they do not arise from an operator calculus; they enumerate what practitioners found useful.

From pattern reuse to search /ML based generation

- Phase 1–4 introduce **search based (PSO, GP, GEP, GA)**[25,26,27,29,31,32] and **ML based prediction/mining** [40,45,46,47,52,55,38] methods:
 - Search methods work over **explicit syntactic templates** (polynomials, symbolic expressions, MR expression trees, assertion templates).
 - ML methods map **structural or textual features** to MR labels.
- In all these, the **search space is defined syntactically** or via patterns; operators are “language constructs” (AST nodes, numeric operations), not semantic operators with a defined algebra.

Model and structure driven MR generation (but not algebraic)

- **Relational algebra** is used as a *reference language* for MRPs in QBSs [12,19], but there is no formal algebraic closure or boundary analysis.
- **Input pattern structural models** [17] provide decomposition theorems for input patterns but do not extend to full operator algebras or MR class boundaries.
- **Causal graphs** [34] and **abstraction/refinement loops** [44] add formal structure, but in terms of **causality and abstraction**, not algebraic operator laws.

From automation effectiveness to structural adequacy and coverage

- Most work evaluates MRs via **fault detection**, mutant kill rate, and code/branch coverage [25,26,31,32,38].
- Adequacy work [7] begins to **formalize MT-specific adequacy criteria**, but still largely in terms of **input/MR usage and code coverage**, not operator block coverage.
- NOETHER [1] introduces **operator block coverage and algebraic structural adequacy** as explicit metrics, decoupling:
 - “How many faults are detected?” from
 - “Which operator equivalence laws and blocks of the operator algebra are exercised?”

Influential Research Clusters and Their Trajectories

Chen–Zhou–Sun cluster (Hong Kong & collaborators): category choice, METRIC(+), CCML, MT, surveys

- **Key works:** METRIC [8], METRIC+ [10,11], MT and MT extension [20,22], MR pattern work [21], MT/MT for LLM surveys [6,2,3,9], CCML/CCMT [13], ML prediction (through references), LLM capability study [51].
- **Contributions:**
 - Established MT and MR identification as core topics via **highly cited surveys** [5,6,2,3].
 - Developed **category choice frameworks** for MR identification (METRIC, METRIC+) and later a **general MR description language (CCML)** and tool (CCMT) [13].
 - Pioneered **data mutation based MR guidance (MT)** [20,22] and MR patterns for enhancing system use [21].
 - Recently, systematically evaluated **LLMs’ ability to discover MRs** [51] and surveyed MT–LLM interactions [9].
- **Trajectory:**
 - From **manual MRs and conceptual surveys** ‘structured pattern/category choice methods’ template /data guided MR generation ‘MR languages and LLM support.
 - Philosophically, they treat MR identification as a **knowledge engineering problem** (patterns, categories, templates) augmented by automation.

Segura–Ruiz Cortés–Troya cluster (Seville & collaborators): MR patterns, QBS, model transformations, MR generators

- **Key works:** MT survey [5], MRPs for QBS [12], MROPs for REST [15], likely MR inference for model transformations [37,39], automated MR generation for QBSs [19].
- **Contributions:**
 - Early champions of **MR pattern catalogues** and **domain specific taxonomies** especially for query based and RESTful systems.
 - Introduced **MRIPs/MROPs** and MR pattern mining from traces [37], plus automated QBS MR generators [19].
- **Trajectory:**
 - Strong emphasis on **making MR identification systematic and reusable** within domains by cataloguing patterns and mining from execution traces.

- Their work is central to the **pattern/taxonomy** axis identified in later surveys [2,3].

Kanewala & collaborators: ML prediction of MR labels

- **Key works:** ML for MR detection [46], graph kernel prediction [47,52], semi supervised MR prediction [49].
- **Contributions:**
 - Established MR identification as a **classification problem over program representations** (CFGs, graph kernels).
 - Demonstrated that **structural code properties** can predict which MRs hold, reducing expert effort.
- **Trajectory:**
 - Continues to influence later predictive work (e.g., Duque Torres et al. [55]) and ML based MR tools referenced in surveys [2,4,7].

Ayerdi-Tonella & CPS cluster: GP based MR generation

- **Key works:** GAssertMRs [32], GenMorph [31].
- **Contributions:**
 - Pushed **genetic programming** as a practical MR generator for both industrial CPS and library code.
 - Careful empirical analysis of mutation score, false positives/negatives, etc.
- **Trajectory:**
 - Define a strong **search based MR generation**line that is now a standard baseline (outperforming earlier tools like AutoMR [26] in some settings [31]).

Xu-Cheung & co.: MR mining from tests and LLM assisted transformation recovery

- **Key works:** MR Scout [38], MR Adopt [41].
- **Contributions:**
 - Recognize **developer tests as a rich MR source** [38].
 - Use LLMs to reconstruct reusable input transformations from single MRs [41].
- **Trajectory:**
 - Shift from **mining implicit human knowledge** (test suites) 'augmenting it with LLM generation and refinement, blending static and semantic inference.

Li-nuclear/NOETHER cluster: domain hierarchies and operator algebraic NOETHER

- **Key works:** Nuclear MR hierarchical models [56,18], NOETHER [1].
- **Contributions:**
 - First, **domain specific MR hierarchies**for nuclear codes, emphasizing **physics/algorithm/code levels** and the distinction between static and dynamic MRs [56,18].
 - Then, **NOETHER** [1] as the first **operator algebraic MR identification framework** explicitly building MetaPatterns from an operator algebra with formal closure and decidability.
- **Trajectory:**
 - Clear evolution from **domain structured MR provenance**(hierarchies) 'general operator algebraic MR derivation making them the central contributors to the operator algebra perspective.

From Adjacent Structures to Operator Algebras: Precursors and Distinctions

Several strands approach algebraic ideas without crossing into a true **operator algebraic MR derivation**

- **Relational algebra as inspiration:** MR patterns for QBS [12,19] talk about relational algebra, but:
 - Operators are used to **describe queries**, not to build a **closed algebra with identities/equivalences** that generates MRs.
 - Patterns are still enumerated and validated empirically.
- **Structural decompositions of input patterns:**
 - Yan et al. [17] define **classes of basic input patterns** and prove that any input pattern decomposes into them, with an identification algorithm.
 - This is algebraic in flavor, but:
 - Only targets **input patterns**, not full input-output operator behavior.

- Does not map these patterns to **equivalence preserving operator transformations** nor identify **numerical or domain conditions** where they fail.
- **Tabular expressions and model transformations** [16,37,39]:
 - Treat specs or traces as structured objects from which MRs can be derived systematically.
 - They do not introduce an **operator algebra over program transformations**; the derived MRs are still pattern instances.
- **Causal graphs** [34] and **abstraction/refinement** [44]:
 - Provide **graphical or logical structure** to generate candidate MRs, especially for scientific and causal software.
 - They are about **structural/causal relations**, not equivalence laws in an algebra.

None of these efforts:

1. Define a set of **primitive operators over a program family**, with composition and equivalence laws.
1. Use those laws to systematically **derive classes of MRs** as consequences of operator equivalences or monotone transformations.
1. Provide **explicit boundary characterizations** (where the laws fail) and **coverage notions over operators/operator blocks**.

NOETHER [1] is the first in this corpus to satisfy all three, making the operator algebraic dimension **orthogonal** to:

- Expert/manual MRs (including domain theory based ones).
- Taxonomy/pattern based reuse.
- Search/ML/LLM based MR generation from traces, code, or text.

Implications for the Field and NOETHER's Positioning

Historical narrative

1. **Manual era ' pattern/taxonomy**
 - MRs begin as **expert invented semantic constraints** later organized into **families and patterns** [5,6,12,14,15,21,37].
1. **Automation era (search/ML/mining):**
 - MR identification becomes a **search/learning problem** over syntactic templates, code graphs, or test artifacts [23,25,26,31,32,38,40,45,46,47,52,55].
 - These methods primarily optimize **fault detection** and **coverage over code/test inputs**, not structural adequacy in an operator space.
1. **Structure aware era (patterns + models + LLMs)**
 - MR identification is enriched with **causal models** [34], **specification languages** [13], **abstraction/refinement** [44], and **LLM driven synthesis** [9,41,42,43,51].
 - Formal structure is still **pattern , graph , or model based** not operator algebraic.
1. **Operator algebraic turn (NOETHER)**
 - NOETHER [1] brings a **compositional operator algebra** to the center, with:
 - Declared operator blocks and scope preconditions.
 - Formal closure and decidability theorems.
 - MetaPatterns as equivalence classes over MRs with **traceable algebraic origin, documented boundaries, and operator block coverage**
 - It treats MR identification as **constructing and exploring the operator algebra** underlying a program family, rather than as:
 - Enumerating MR patterns.
 - Searching over syntactic templates.
 - Predicting MR labels via ML.

Future directions suggested by the trajectory

- **Integration of operator algebras with existing methods:**
 - Pattern/taxonomy work could be **re grounded** in an operator algebra (derive MRPs/MROPs as algebraic classes) and given explicit boundary conditions.
 - Search based generators (GenMorph, AutoMR, GEP) could restrict or guide their search to **algebraically justified regions** of the MR space.

- **Novel adequacy metrics:**
 - Current adequacy notions focus on **input/test and code coverage** [7].
 - Operator algebraic frameworks like NOETHER enable **coverage over operators, compositions, and equivalence laws**, yielding a new axis of test set adequacy.
- **LLMs with algebraic constraints:**
 - LLMs are currently used to generate MR descriptions and transformations [41,42,43,51]; an operator algebra could provide them with a **formal bias**:
 - Prompting them to propose transformations that correspond to **specific operator blocks**.
 - Automatically checking whether generated MRs can be justified by known algebraic laws.
- **Multi dimensional evidence for MR sets**
 - The field has largely equated MR quality with **fault detection** (mutation scores, bug counts).
 - NOETHER's framing [1] opens up **distinct evidence dimensions**:
 - **Algebraic origin and soundness** (which laws underpin each MR).
 - **Boundary conditions** (numeric, domain, or operator configurations where MRs do or do not hold).
 - **Coverage of the operator algebra**.
 - This allows future work to **separate structural adequacy from detection power**, and to argue for complementary MT approaches rather than purely competitive ones.

In this historical context, NOETHER can be precisely framed as a **complementary MR identification method** whose key novelty is deriving **MR classes and boundaries from an explicit operator algebra**, adding **operator block coverage and algebraic provenance** as first class evidence dimensions beyond the fault detection metrics that dominate previous generations of MT research.

Foundational Work

Which papers form the foundational references on this topic?

The below table shows the resources that are most often cited by the relevant papers on this topic. This is measured by the **reference rate**, which is the fraction of relevant papers that cite a resource. Use this table to determine the most important core papers to be familiar with if you want to deeply understand this topic. Some of these core papers may not be directly relevant to the topic, but provide important context.

Ref.	Reference Rate	Title	Cited By These Relevant Papers
[47]	0.48	Predicting metamorphic relations for testing scientific software: a machine learning approach using graph kernels	[2, 3, 4, 5, 6, 7, 10, 12, 15, 16, 17, 19, 21, 22, 24, 26, 29, 31, 34, 37, 38, 42, 44, 45, 49, 51, 52, 55, 57, 60, 63, 64, 67, 70, 71, 72, 83]
[25]	0.47	Search-based inference of polynomial metamorphic relations	[2, 3, 4, 5, 6, 10, 17, 19, 20, 22, 23, 24, 26, 27, 28, 29, 31, 32, 33, 34, 37, 38, 40, 41, 42, 43, 47, 51, 52, 53, 55, 57, 64, 67, 72, 75, 90, 91, 92]
[46]	0.43	Using machine learning techniques to detect metamorphic relations for programs without test oracles	[2, 3, 4, 5, 8, 10, 12, 19, 20, 22, 23, 24, 25, 26, 27, 28, 29, 31, 35, 37, 38, 40, 42, 47, 49, 51, 52, 55, 57, 60, 62, 69, 70, 72, 81]
[8]	0.36	METRIC: METAmorphic Relation Identification based on the Category-choice framework	[2, 3, 4, 5, 6, 7, 10, 13, 15, 16, 22, 26, 29, 33, 34, 38, 42, 51, 52, 55, 57, 63, 64, 67, 71, 79, 82, 92]
[58]	0.34	Metamorphic Testing: A New Approach for Generating Next Test Cases	[2, 3, 4, 6, 9, 10, 15, 19, 22, 26, 31, 32, 37, 38, 40, 42, 48, 51, 52, 60, 62, 68, 76, 82, 84, 88, 90, 91, 94]
[5]	0.33	A Survey on Metamorphic Testing	[2, 3, 4, 6, 7, 9, 10, 13, 14, 15, 29, 31, 32, 33, 37, 38, 40, 42, 44, 51, 68, 76, 88, 90, 94]
[26]	0.31	Automatic Discovery and Cleansing of Numerical Metamorphic Relations	[2, 3, 4, 7, 17, 27, 28, 31, 33, 38, 40, 41, 42, 51, 53, 67, 73, 77, 82, 91]
[32]	0.25	Generating metamorphic relations for cyber-physical systems with genetic programming: an industrial case study	[2, 3, 4, 19, 22, 31, 38, 41, 43, 51, 72, 90, 92]

[40]	0.25	MeMo: Automatically identifying metamorphic relations in Javadoc comments for test automation	[2, 3, 7, 19, 22, 31, 33, 38, 42, 51, 72]
[2]	0.24	Metamorphic Relation Generation: State of the Art and Research Directions	[9, 13, 82, 84, 90]
[6]	0.23	Metamorphic Testing	[2, 13, 14, 22, 29, 32, 33, 38, 42, 44, 48, 51, 62, 68, 82, 88, 90, 94]
[37]	0.23	Automated inference of likely metamorphic relations for model transformations	[2, 3, 4, 12, 13, 15, 16, 19, 21, 22, 29, 32, 33, 40, 72]
[141]	0.22	The Oracle Problem in Software Testing: A Survey	[2, 3, 4, 5, 6, 8, 15, 19, 22, 29, 31, 32, 33, 37, 40, 51]
[15]	0.20	Metamorphic Testing of RESTful Web APIs	[2, 3, 10, 19, 21, 22, 26, 29, 32, 38, 67, 73, 76, 84, 92]
[31]	0.20	GenMorph: Automatically Generating Metamorphic Relations via Genetic Programming	[2, 13, 41, 48, 51]
[74]	0.20	A New Method for Constructing Metamorphic Relations	[2, 3, 4, 5, 6, 8, 10, 20, 22, 25, 26, 29, 40, 47, 51, 52]
[23]	0.18	Dynamic Inference of Likely Metamorphic Properties to Support Differential Testing	[5, 12, 15, 17, 21, 26, 29, 37, 40, 49, 52, 60, 67]
[51]	0.17	Can Large Language Models Discover Metamorphic Relations? A Large-Scale Empirical Study	[2, 9, 42]
[38]	0.17	MR-Scout: Automated Synthesis of Metamorphic Relations from Existing Test Cases	[2, 41, 42, 48, 51]
[142]	0.17	On Testing Non-Testable Programs	[4, 8, 15, 19, 20, 25, 26, 37, 40, 46, 47, 52, 58]

Adjacent Work

Which papers cite the same foundational papers as relevant papers?

Use this table to discover related papers on adjacent topics, to gain a broader understanding of the field and help generate ideas for useful new research directions.

Ref.	Adjacency score	Title	References These Foundational Papers
[154]	5.69	A metamorphic relation recommendation method utilizing program syntax and semantic similarity	[8, 22, 25, 29, 31, 40, 46, 47, 58]
[129]	5.28	MR-Coupler: Automated Metamorphic Test Generation via Functional Coupling Analysis	[5, 8, 15, 20, 25, 26, 31, 38, 40, 51]
[155]	4.12	SEDMR: A spreadsheet error detection approach based on metamorphic testing	[4, 5, 6, 8, 20, 46, 47]
[106]	4.08	Using Metamorphic Relations in Redundancy-based Fault/Intrusion Tolerance	[2, 5, 19, 31, 32, 46, 47, 58, 64]
[156]	3.91	Cross-Domain Multi-Label Prediction of Metamorphic Relation Patterns Leveraging Multimodal Features	[5, 6, 19, 37, 46, 47, 58]
[157]	3.84	Identifying metamorphic relations A data mutation directed approach	[8, 20, 29, 32, 37, 40]
[158]	3.68	The Future of AI-Driven Software Engineering	[25, 26, 31, 32, 38, 40]
[159]	3.54	Boosting Multimode Ruling in DHR Architecture With Metamorphic Relations	[8, 20, 25, 40, 46, 47]
[160]	3.53	Can Metamorphic Relations be Automatically Discovered?	[8, 20, 26, 32, 40, 46]
[161]	3.40	Using Category Partition to Detect Metamorphic Relations	[15, 20, 25, 38, 40, 47]
[162]	3.38	On the effectiveness of testing sentiment analysis systems with metamorphic testing	[15, 20, 25, 32, 46, 47]
[163]	3.30	Fuzzing deep-learning libraries via automated relational API inference	[25, 26, 29, 37, 40]
[164]	3.20	Predicting Metamorphic Relations for Matrix Calculation Programs	[8, 23, 25, 46, 47]
[165]	3.14	Test Oracle Automation in the Era of LLMs	[15, 31, 33, 38, 40]

[107]	2.36	MRGS ART: Metamorphic Relation and Group Selection Based on Adaptive Random Testing	[6, 8, 19, 25, 47]
[90]	2.07	MDPMorph: An MDP-Based Metamorphic Testing Framework for Deep Reinforcement Learning Agents	[2, 5, 6, 25, 32, 58]
[138]	2.04	Path-directed source test case generation and prioritization in metamorphic testing	[8, 15, 20, 26]
[110]	2.00	Selecting and Constraining Metamorphic Relations	[15, 40, 46, 47]
[147]	1.93	Leveraging mutants for automatic prediction of metamorphic relations using machine learning	[46, 47, 52]
[92]	1.92	Feedback-Directed Metamorphic Testing	[8, 15, 20, 25, 32]

References

- [1] [NOETHER: A Constructive Framework for Metamorphic Pattern Discovery from Operator Algebras](#)
Meng Li, ..., and China. ArXiv, 2026. 0 citations.
100% Topic Match
Proposes a constructive operator algebraic framework (NOETHER) for mechanically deriving metamorphic patterns. Implements an eight block operator decomposition upstream and a CONSTRUCT MP algorithm downstream with algebraic closure (Thm1) and polynomial time decidability (Thm2). Explicitly ties each MetaPattern to algebraic origin and scope preconditions, defines out of scope boundaries, reports domain case studies (Boltzmann reactor, equivariant ML), empirical comparisons to GP baselines, and documents falsifications and extension directions — so qualifies as an operator algebraic MR derivation method rather than taxonomy or search only.
- [2] [Metamorphic Relation Generation: State of the Art and Research Directions](#)
Rui Li, ..., and Tsong Yueh Chen. ACM Transactions on Software Engineering and Methodology, 2024. 11 citations.
100% Topic Match
Surveys the state of the art in metamorphic relation (MR) generation. Reviews and classifies existing MR generation methods from literature, synthesizing trends and research gaps. Primarily a taxonomy/systematic review: does not present an operator algebraic derivation of MR classes or formal operator level coverage; useful for locating expert driven, taxonomy based, and search/ML approaches and gaps.
- [3] [Metamorphic Relation Generation: State of the Art and Visions for Future Research](#)
Rui Li, ..., and Tsong Yueh Chen. ArXiv, 2024. 6 citations.
100% Topic Match
Summarizes state of the art MR generation techniques (systematic review). Surveys taxonomy/pattern, search/ML, execution mining and miscellaneous MR identification approaches and highlights future research directions. Does not present an operator algebraic derivation of MR classes or formal operator level coverage; treats many methods as template/black box and notes numeric/template limitations.
- [4] [Metamorphic relation automation: Rationale, challenges, and solution directions](#)
Emran Altamimi, ..., and C. Catal. Journal of Software: Evolution and Process, 2022. 4 citations.
100% Topic Match
Presents a systematic literature review on automating metamorphic relation (MR) identification. Synthesizes existing MR automation techniques from papers, categorizing methods (CFG/PDG analysis, execution trace mining, ML, seed MR use). Notes automation works best in mathematical/scientific domains; treats MR generation as early stage research without operator algebraic derivation or explicit MR origin/boundary/coverage claims.
- [5] [A Survey on Metamorphic Testing](#)
Sergio Segura, ..., and Antonio Ruiz-Cortés. IEEE Transactions on Software Engineering, 2016. 532 citations.
100% Topic Match
Provides a comprehensive survey of metamorphic testing practice and MR examples. Reviews MR definitions, taxonomy of MR types, applications, empirical studies, and techniques aiding MR identification (symbolic execution, search). Does not present a formal operator/algebra that systematically derives MR classes or explicit MR-origin/boundary/ operator coverage metrics; categorization is descriptive/taxonomy style.
- [6] [Metamorphic Testing](#)
T. Chen, ..., and Z. Zhou. ACM Computing Surveys (CSUR), 2018. 394 citations.
100% Topic Match
Provides a comprehensive survey of metamorphic testing and metamorphic relations. Reviews MT definitions, workflows, applications, challenges, and integrates prior studies into a unified framework. Does not present an operator algebraic derivation of MR classes, nor explicit MR provenance, boundary characterization, or operator level coverage metrics (taxonomic/expert focus).
- [7] [Test Adequacy for Metamorphic Testing: Criteria, Measurement, and Implication](#)
An Fu, ..., and Huai Liu. ArXiv, 2024. 0 citations.
100% Topic Match
Proposes new test adequacy criteria and a measurement framework for metamorphic testing. Frames MT formally ($f: X^*Y$, $MR \text{ as } R \uparrow X^n \times Y^n$), defines adequacy metrics, and reports an empirical study. Surveys and explicitly categorizes MR identification methods (pattern/template, ML, category/taxonomy, search, datamorphic, NLP), but does not derive MRs from an operator algebra or provide operator block coverage/origin boundaries.
- [8] [METRIC: METamorphic Relation Identification based on the Category-choice framework](#)
T. Chen, ..., and Xiaoyuan Xie. J. Syst. Softw., 2016. 80 citations.
100% Topic Match
No summary or abstract available
- [9] [Bidirectional Empowerment of Metamorphic Testing and Large Language Models: A Systematic Survey](#)
Zheng Zheng, ..., and Tsong Yueh Chen. ArXiv, 2026. 0 citations.
100% Topic Match
Surveys bidirectional interactions between metamorphic testing and LLMs. Systematically reviews 93 studies, categorizing MT-for-LLMs and LLMs-for-MT, and summarizes LLM-assisted MR discovery tools and techniques. Focus is on automation of MR discovery (LLM prompting, mining, synthesis); does not claim deriving MR classes from an explicit operator algebra with formal origins, boundaries, or operator-block coverage.
- [10] [METRIC⁺: A Metamorphic Relation Identification Technique Based on Input Plus Output Domains](#)
Chang-ai Sun, ..., and T. Chen. IEEE Trans. Software Eng., 2021. 48 citations.
100% Topic Match
Proposes an enhanced MR identification method (METRIC+) that incorporates output domain information. Extends prior METRIC (which used input domain/category choices) by analyzing outputs from complete test frames and implements a tool; evaluated on four specs with experiments and comparisons to MT. This is a taxonomy/template/search style MR generator (extends an existing framework); it does not present an operator algebraic derivation of MR classes, nor explicit algebraic origin, boundaries, or operator block coverage.
- [11] [METRIC⁺: A Metamorphic Relation Identification Technique Based on Input P](#)
Chang-ai Sun, ..., and T. Chen. IEEE Transactions on Software Engineering, 2021. 0 citations.
100% Topic Match

- Proposes an enhanced, input and output aware MR identification method (METRIC+).
Extends prior METRIC (category choice, input domain focused) by incorporating output domain information to identify metamorphic relations from complete test frames.
Appears taxonomy/template/search style (practical MR extraction from test data); no evidence of deriving MR classes from a formal operator algebra, explicit algebraic origins, or operator block coverage.
- [12] [Metamorphic Relation Patterns for Query-Based Systems](#)
Sergio Segura, ..., and Antonio Ruiz-Cortés. 2019 IEEE/ACM 4th International Workshop on Metamorphic Testing (MET), 2019. 22 citations.
100% Topic Match
Proposes a catalogue of metamorphic relation patterns (MRPs) for query based systems.
Defines MRPs by mapping common query features to relational algebra rooted templates and demonstrates them on PrestaShop.
Patterns are presented as reusable templates/abstract MR classes (intended to aid systematic identification/automation) but do not appear to formalize an operator algebra with composition laws, explicit MR origins/boundaries, or operator block coverage metrics.
- [13] [Boosting Metamorphic Testing: A General Metamorphic Specification Language and A Supporting System](#)
Chang-ai Sun, ..., and Pak-Lok Poon. ACM Transactions on Software Engineering and Methodology, 2026. 0 citations.
100% Topic Match
Proposes a general MR specification language (CCML) and supporting MT tool (CCMT).
Implements CCML to express MRs, automatically generate test inputs, compose MRs, and optimize runtime; evaluated empirically across domains.
Likely taxonomy/template driven and tool centric: paper promotes a specification language and automation, but doesn't claim deriving MR classes from an operator algebra nor traceable algebraic origins or operator block coverage—check full text for any formal operator/compositional algebra or explicit MR provenance/boundary claims.
- [14] [Metamorphic Relation Patterns for Metamorphic Testing, Exploration and Robustness](#)
Zhihao Ying, ..., and Zhi Quan Zhou. Software Testing, 2025. 2 citations.
100% Topic Match
Proposes a taxonomy-style framework of metamorphic relation patterns (MRPs) and MRP family trees.
Defines formal relations among MRPs, identifies new MRPs, and organizes both into two family trees guiding MR identification and application.
Likely taxonomy based/manual: no evidence here of deriving MR classes from an explicit operator algebra, formal operator origins, boundary conditions, or operator block coverage metrics.
- [15] [Metamorphic Testing of RESTful Web APIs](#)
Sergio Segura, ..., and Antonio Ruiz-Cortés. IEEE Transactions on Software Engineering, 2018. 138 citations.
100% Topic Match
Proposes a taxonomy of six abstract Metamorphic Relation Output Patterns (MROPs) for RESTful Web APIs.
Defines each MROP formally as a pattern (S, F1, F2, ..., Fn) and instantiates 60 concrete MRs across six real APIs, then evaluates them experimentally.
MR identification is taxonomy/pattern based and manual instantiation; no operator algebraic derivation, no formal MR origin/boundary proofs, and no operator level coverage metric.
- [16] [Tabular expression based method for constructing metamorphic relations](#)
Jingyao Li, ..., and Peng Zhang. Software: Practice and Experience, 2020. 6 citations.
100% Topic Match
Proposes a tabular expression based method for constructing metamorphic relations.
Extracts MRs systematically from specification tables by mapping header-main grid relationships; claims domain independent, reusable MR templates.
MR origin is specification driven (not operator algebraic); evaluates effectiveness on five apps and compares fault detection, but does not derive MRs from an operator calculus or provide operator level coverage/boundary proofs.
- [17] [Identification Algorithm Framework and Structural model on Input Pattern of Metamorphic Relations](#)
Shiyu Yan, ..., and Jie Liu. 2022 9th International Conference on Dependable Systems and Their Applications (DSA), 2022. 1 citations.
100% Topic Match
Proposes an algorithmic framework and structural model for identifying input patterns of metamorphic relations.
Shows this by defining "basic input pattern" classes, proving any input pattern composes from them for multi variate numeric functions, and giving an identification algorithm with complexity analysis.
Relevant: focuses on MR identification (input pattern decomposition) and complexity reduction; does NOT appear to derive MRs from an operator algebra or provide algebraic origin/boundary/coverage over program operators—thus classifies as structural/taxonomy/algorithmic, not operator algebraic.
- [18] [A Lightweight Verification Method Based on Metamorphic Relation for Nuclear Power Software](#)
Meng Li, ..., and Jun Sun. Unknown journal, 2022. 1 citations.
100% Topic Match
Proposes a lightweight metamorphic testing verification method for nuclear power software.
Identifies MRs from physical equations, numerical algorithms, and program specifications, and applies them at unit/integration/system levels to detect algorithmic vs. model errors.
Offers an MR hierarchical classification (physics model, computational model, code model, "likely" MR) and distinguishes static (equation/algorithm derived) vs. dynamic (execution/data fitted) MR identification; does NOT define an operator algebra, composition/equivalence laws, or operator level coverage metrics.
- [19] [Automated Generation of Metamorphic Relations for Query-Based Systems](#)
Sergio Segura, ..., and Antonio Ruiz-Cortés. 2022 IEEE/ACM 7th International Workshop on Metamorphic Testing (MET), 2022. 14 citations.
100% Topic Match
Proposes an automated method to generate metamorphic relations for query based systems.
Derives MRs from a lightweight specification of query parameters and searches/query semantics to produce many MR instances automatically.
Focuses on QBS domains (IMDb, SkyScanner, YouTube); this is a search/template based MR generation approach, not an operator algebraic derivation—paper does not claim algebraic operator model, formal MR origins, or operator block coverage.
- [20] [MT: A Data Mutation Directed Metamorphic Relation Acquisition Methodology](#)
Chang-ai Sun, ..., and W. Chan. 2016 IEEE/ACM 1st International Workshop on Metamorphic Testing (MET), 2016. 27 citations.
100% Topic Match
No summary or abstract available
- [21] [Metamorphic Relations for Enhancing System Understanding and Use](#)
Z. Zhou, ..., and Dave Towey. IEEE Transactions on Software Engineering, 2020. 115 citations.
100% Topic Match
Proposes reusable MR patterns ("symmetry" and "change direction") for deriving concrete metamorphic relations.
Presents pattern definitions plus examples and large empirical studies across 65 websites and multiple services to derive and validate MRs.
Focus is pattern/taxonomy-style MR design for user understanding; does NOT formulate an operator algebra, nor track algebraic origin, formal boundaries, or operator-block coverage.
- [22] [Identifying metamorphic relations: A data mutation directed approach](#)
Chang-ai Sun, ..., and W. Chan. Software: Practice and Experience, 2023. 4 citations.
100% Topic Match
Proposes a data-mutation-directed method (MT) to support MR identification.
Provides mutation operators plus template-style mapping rules and a tool; guides testers to craft MRs and evaluates effectiveness and fault-detection.

Manual/tester-in-the-loop approach: templates + mutations are search/assisted, not an operator-algebra derivation; provenance, MR boundaries, or operator-block coverage are informal.

[23] [Dynamic Inference of Likely Metamorphic Properties to Support Differential Testing](#)

Fang-Hsiang Su, ..., and Gail E Kaiser. 2015 IEEE/ACM 10th International Workshop on Automation of Software Test, 2015. 28 citations.
100% Topic Match

Proposes an automatic, dynamic system (KABU) to infer metamorphic properties from executions.
Instruments methods, records arguments/returns, applies primitive input transformers (templates), and mines relations from traces; supports metamorphic differential testing across versions.
Relevant as a search/trace mining MR approach (learned/templates from execution); contains no operator algebra derivation, explicit MR origin/boundary proofs, or operator block coverage metrics.

[24] [ARCAMETES: A Learning Approach for Metamorphic Exploration and Testing](#)

Darryl C. Jarman, ..., and Z. Zhou. Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops, 2020. 1 citations.
100% Topic Match

Proposes a learning-based method for discovering metamorphic relations.
Uses association-rule learning plus combinatorial testing to mine candidate MRs from program executions and input-output patterns.
Falls into search/ML-generated MR identification; does not claim an operator-algebraic derivation, formal MR origins/boundaries, or operator-block coverage.

[25] [Search-based inference of polynomial metamorphic relations](#)

Jie M. Zhang, ..., and Hong Mei. Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering, 2014. 118 citations.
100% Topic Match

No summary or abstract available

[26] [Automatic Discovery and Cleansing of Numerical Metamorphic Relations](#)

Bo Zhang, ..., and P. Moscato. 2019 IEEE International Conference on Software Maintenance and Evolution (ICSME), 2019. 44 citations.
100% Topic Match

Proposes an automated, search based method for discovering and cleansing numerical MRs.
Uses particle swarm optimization to infer equality/inequality MRs, then SVD and constraint solving to prune redundancies.
MR discovery is template/search driven for numerical programs (no operator algebra derivation, provenance/boundary claims limited; evaluated by fault detection).

[27] [A Metamorphic Relation Identification Method based on GEP](#)

Jie Hong, ..., and Helin Gong. 2022 9th International Conference on Dependable Systems and Their Applications (DSA), 2022. 1 citations.
100% Topic Match

Proposes an automated MR identification method using gene expression programming (GEP).
Converts MR discovery into symbolic expression regression: generate input pairs, mine output relations via GEP, reduce and verify candidates.
Classified as search/ML based MR generation (template/regression search); no operator algebra, provenance, boundaries, or operator coverage analysis provided.

[28] [A Dynamic Recognition Method of Metamorphic Relation Identification](#)

Jie Hong, ..., and Helin Gong. 2022 13th International Conference on Reliability, Maintainability, and Safety (ICRMS), 2022. 4 citations.
100% Topic Match

Proposes an automatic MR identification method via symbolic regression.
Transforms MR recognition into symbolic expression regression: generate input pairs by preset patterns, mine output relations with gene expression programming, then reduce and verify candidate MRs.
This is a search/learning approach (GEP on traces); it does NOT define an operator algebra, nor trace MR origins/boundaries or operator block coverage — evaluates effectiveness vs. static analysis and prior automatic methods.

[29] [A Genetic Algorithm-Based Approach for Composite Metamorphic Relations Construction](#)

Zhenglong Xiang, ..., and Fei Yu. Inf., 2019. 12 citations.
100% Topic Match

Proposes an automated, search based method to construct composite metamorphic relations.
Represents composite MRs as sequences of primitive relations and uses a genetic algorithm with dynamic executions to search for high quality sequences.
Classified as search/ML style MR identification: uses predefined relation primitives and empirical fault detection evidence; it does NOT define an operator algebra, state composition/equivalence laws, derive MR classes/boundaries from such an algebra, or provide operator block coverage or traceable algebraic origins.

[30] [A Study on the Number of Composite Layers and Effectiveness of Metamorphic Relation](#)

Xueyue Zhang, ..., and Shiyu Yan. 2023 IEEE International Conference on Electrical, Automation and Computer Engineering (ICEACE), 2023. 2 citations.
100% Topic Match

Proposes a study on how composing metamorphic relations (MRs) into multi-layer composites affects effectiveness and cost.
Defines a "combined operation" for numerical-expression MRs, empirically measures fault-detection vs. computational cost across composite layer counts, and finds three layers most cost effective.
Relevant as search/generated MR work focused on MR composition and empirical cost effectiveness for numerical MRs; does NOT present an operator algebraic derivation of MR classes, formal origins, boundaries, or operator block coverage.

[31] [GenMorph: Automatically Generating Metamorphic Relations via Genetic Programming](#)

J. Ayerdi, ..., and Paolo Tonella. IEEE Transactions on Software Engineering, 2023. 20 citations.
100% Topic Match

Proposes an automated, search based method (GenMorph) to generate metamorphic relations.
Uses genetic programming guided by two fitness functions minimizing false alarms and missed faults to evolve executable MRs for Java methods.
Focuses on boolean/numeric/ordered sequence I/O, frames MR discovery as search/ML rather than operator algebraic derivation; does not claim operator algebra, provenance, or operator block coverage.

[32] [Generating metamorphic relations for cyber-physical systems with genetic programming: an industrial case study](#)

J. Ayerdi, ..., and Maite Arratibel. Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2021. 56 citations.
100% Topic Match

Proposes an automated, search based method to generate metamorphic relations using genetic programming.
Implements GAssertMRs: a multi objective GP over MR templates (MRIPs + output templates) using labeled execution traces and optional mutants to evolve MRs minimizing false positives/negatives and size.
Relevant as search/ML based MR identification (template-driven), not operator algebraic: MRs come from curated templates and data, provenance/boundaries and operator block coverage are not formalized.

[33] [Abstraction-Aware Inference of Metamorphic Relations](#)

Agustín Nolasco, ..., and M. Frias. Proceedings of the ACM on Software Engineering, 2024. 6 citations.
100% Topic Match

Proposes an automated MR inference method (MemoRIA) focused on method/method-sequence equivalences.
Builds object-protocol abstractions, fuzzes the abstraction to generate candidates, validates via runtime analysis, and prunes redundants with SAT.

Relevant as a search/automatic MR generation paper: it does not define an operator algebra over a program family nor trace MRs to algebraic laws or operator block coverage.

[34] [Metamorphic Testing with Causal Graphs](#)

Andrew G. Clark, ..., and R. Hierons. 2023 IEEE Conference on Software Testing, Verification and Validation (ICST), 2023. 9 citations.
100% Topic Match

Proposes an automated, causal-graph-based method to generate metamorphic relations and tests.
Uses causal DAGs to specify expected input-output causal relationships, deriving MRs and source/follow-up cases; evaluates robustness and bug detection on controlled experiments and Defects4J case studies.
Does not develop an operator algebraic derivation of MR classes or operator block coverage; classifies MR generation as model based (causal), contrasting with manual, taxonomy, and search/ML approaches.

[35] [Automated identification of metamorphic test scenarios for an ocean-modeling application](#)

D. J. Hiremath, ..., and W. Rath. 2020 IEEE International Conference On Artificial Intelligence Testing (AITest), 2020. 6 citations.
100% Topic Match

Proposes an automated, search/ML method to generate metamorphic input transformations (g) for an ocean model.
Implements g as affine transformations on input points and uses an optimization/search (genetic/Monte Carlo style) cost that penalizes identity and later rewards orthogonality to previously found g.
Does not develop an operator algebra or formal MR provenance/boundary/coverage; this is a template/search based approach with manual interpretation of discovered MRs relative to physical symmetries.

[36] [Towards Automated Metamorphic Test Identification for Ocean System Models](#)

D. J. Hiremath, ..., and W. Rath. 2021 IEEE/ACM 6th International Workshop on Metamorphic Testing (MET), 2021. 7 citations.
99% Topic Match

Proposes an automated, ML based method to identify metamorphic relations for ocean system models.
Represents MRs as (initially) identity maps, defines a cost to find MRs orthogonal to previously found ones and penalize trivial identities, uses ML optimization plus dimensionality reduction and targeted mutation to reduce complexity.
Domain uses known physical symmetries as context; method is search/ML based (not operator algebraic), work in progress with assumptions (h = identity), MR decomposition needed, and artifacts/data are available.

[37] [Automated inference of likely metamorphic relations for model transformations](#)

J. Troya, ..., and Antonio Ruiz-Cortés. J. Syst. Softw., 2018. 47 citations.
99% Topic Match

Proposes an automated trace based method to infer likely metamorphic relations for ATL model transformations.
Extracts MR input/output relation pairs from transformation execution traces using five trace patterns and a catalogue of 24 domain independent MR templates; evaluates precision (96.4%) and mutation score (93.3% in one case).
MR provenance is trace/template based (not operator algebraic); MRs are "likely" (empirically validated) without formal algebraic origin, boundaries, or operator level coverage metrics.

[38] [MR-Scout: Automated Synthesis of Metamorphic Relations from Existing Test Cases](#)

Congying Xu, ..., and S. Cheung. ACM Transactions on Software Engineering and Methodology, 2023. 17 citations.
99% Topic Match

Proposes an automated method to synthesize MRs from existing developer test cases.
Detects MR-encoded test cases, extracts parameterized "codified MRs," filters for quality, and reuses them to generate follow-up tests; evaluated on 701 OSS projects with >11k MTCS.
Positions prior work as manual taxonomy or search/ML-based; does not define an operator algebra or derive MR classes with algebraic origins, boundaries, or operator-block coverage.

[39] [Towards the Automation of Metamorphic Testing in Model Transformations](#)

J. Troya, ..., and Antonio Ruiz-Cortés. ArXiv, 2018. 2 citations.
99% Topic Match

Proposes an automated, trace driven method to generate metamorphic relations for model transformations.
Mines reusable MR patterns from execution trace models and instantiates them (in OCL) using observed transformation traces.
MR identification is pattern/taxonomy and data driven (trace completeness dependent); no operator algebra, formal composition laws, explicit MR origin boundaries, or operator block coverage metrics are provided.

[40] [MeMo: Automatically identifying metamorphic relations in Javadoc comments for test automation](#)

A. Blasi, ..., and A. Carzaniga. J. Syst. Softw., 2021. 58 citations.
99% Topic Match

Proposes an automated method (MeMo) to identify metamorphic equivalence relations from Javadoc.
Extracts and parses natural-language documentation to infer equivalence style MRs, then uses them as oracles in generated tests.
Relevant as a search/ML/text mining MR identification approach (not operator algebraic); documents provenance (doc'MR) and reports effectiveness and completeness with dynamic analysis, but does not derive MR classes from a formal operator algebra or state MR validity boundaries over operator blocks.

[41] [MR-Adopt: Automatic Deduction of Input Transformation Function for Metamorphic Testing](#)

Congying Xu, ..., and Jialun Cao. 2024 39th IEEE/ACM International Conference on Automated Software Engineering (ASE), 2024. 18 citations.
99% Topic Match

Proposes an automatic, example driven method to recover reusable input transformations for encoded MRs.
Uses LLMs to synthesize extra source-follow up pairs from one example, generates candidate transformation code, refines it via data flow analysis, and validates candidates against encoded output relations.
Relevant as a search/ML generated MR identification approach (72% generalization on 100 MRs, coverage and mutation improvements); NOT operator algebraic—no formal operator set, composition laws, MR origins, boundaries, or operator block coverage are derived.

[42] [SVPrompt-MR: An LLM-Based Metamorphic Relation Identification Method with Self-Verification Mechanism](#)

Jun Huang, ..., and Shiyu Yan. 2025 32nd Asia-Pacific Software Engineering Conference (APSEC), 2025. 1 citations.
98% Topic Match

Proposes an LLM-based, self verification prompt method for automatic MR identification.
Uses a two stage pipeline (LLM MR generation + LLM verification) applied to a reactor safety program; compares against expert MRs and prior methods, finding higher coverage and novel MRs.
Relevant as a search/ML/generated MR approach (not operator algebraic); no evidence it defines operators, composition laws, formal MR origins, boundaries, or operator level coverage metrics.

[43] [Towards Generating Executable Metamorphic Relations Using Large Language Models](#)

S. Shin, ..., and Alexandra Baicoianu. Unknown journal, 2024. 12 citations.
98% Topic Match

Proposes an LLM-based method to automatically generate executable metamorphic relations (EMRs).
Uses few-shot prompting with requirements and API specs, plus a Siemens questionnaire and an empirical accuracy evaluation on a web app.
Does not define an operator algebra, composition/equivalence laws, formal MR origins or boundaries, nor any operator-block coverage metric—classify as search/ML-generated MR identification.

[44] [Model Assisted Refinement of Metamorphic Relations for Scientific Software](#)

Clay Stevens, ..., and Myra B. Cohen. 2025 IEEE/ACM 47th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER), 2025. 0 citations.

98% Topic Match

Proposes a model assisted iterative method (MARS) to refine metamorphic relations for scientific software.
Combines domain agnostic initial relations from dynamic analysis with model checking to identify missing I/O patterns, then incorporates expert validation to refine domain specific MRs and tests.
Relevant as a hybrid (dynamic + formal + expert) MR identification workflow—it is not an operator algebraic derivation: MRs originate from dynamic traces and expert refinement, coverage is behavioral (not operator block) and boundaries are validated empirically/formally rather than derived from an operator calculus.

[45] [MRpredT: Using Text Mining for Metamorphic Relation Prediction](#)

Karishma Rahman, ..., and Upulee Kanewala. Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops, 2020. 13 citations.

98% Topic Match

Proposes an ML-based method to predict metamorphic relations from program documentation.
Trains text classification models (compared to a prior graph kernel approach) using textual features of docs to predict MRs for matrix calculation programs.
Relevant as a search/ML MR generation approach (automated, template/feature driven); it does NOT define an operator algebra, nor trace MR origins/boundaries or operator block coverage.

[46] [Using machine learning techniques to detect metamorphic relations for programs without test oracles](#)

Upulee Kanewala and J. Bieman. 2013 IEEE 24th International Symposium on Software Reliability Engineering (ISSRE), 2013. 105 citations.

97% Topic Match

No summary or abstract available

[47] [Predicting metamorphic relations for testing scientific software: a machine learning approach using graph kernels](#)

Upulee Kanewala, ..., and Asa Ben-Hur. Software Testing, 2016. 113 citations.

96% Topic Match

No summary or abstract available

[48] [Metamorphic Testing of Large Language Models for Natural Language Processing](#)

Steven Cho, ..., and Valerio Terragni. 2025 IEEE International Conference on Software Maintenance and Evolution (ICSME), 2025. 10 citations.

95% Topic Match

Presents a large-scale, taxonomy-driven study of metamorphic relations for LLM-based NLP.
Compiles 191 MRs from a literature review, implements 36 representative MRs in LLMORPH, and runs H561K metamorphic tests across three LLMs and four NLP tasks.
Manual/taxonomy selection; documents MR sources and task mappings but does not define an operator algebra, derive MR classes/boundaries from program-family operators, or provide operator-block coverage or algebraic provenance.

[49] [Using Semi-Supervised Learning for Predicting Metamorphic Relations](#)

B. Hardin and Upulee Kanewala. 2018 IEEE/ACM 3rd International Workshop on Metamorphic Testing (MET), 2018. 28 citations.

95% Topic Match

Proposes a semi-supervised ML method to predict applicable metamorphic relations for code.
Trains classifiers on CFG-derived node/path features (Soot ' .dot) for Java methods; unlabeled data improves accuracy versus supervised baseline.
Does not define an operator algebra, composition laws, MR provenance/boundary analysis, or operator-level coverage—this is a search/ML-based MR identification approach.

[50] [RBF-MLMR: A Multi-Label Metamorphic Relation Prediction Approach Using RBF Neural Network](#)

Pengcheng Zhang, ..., and Hareton K. N. Leung. IEEE Access, 2017. 40 citations.

94% Topic Match

Proposes a multi-label MR prediction method using an improved RBF neural network.
Extracts CFG node/path features via Soot, builds multi label datasets, and trains an RBF MLMR classifier with clustering based RBF optimization.
Relevant as a search/ML based MR identification paper (predicts MRs from code features); it does not define an operator algebra, traceable algebraic MR origins, or operator block coverage.

[51] [Can Large Language Models Discover Metamorphic Relations? A Large-Scale Empirical Study](#)

Jiaming Zhang, ..., and Sijin Dong. 2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), 2025. 9 citations.

93% Topic Match

Evaluates LLMs' ability to automatically generate metamorphic relations.
Runs large-scale empirical prompts (GPT-3.5 and likely other LLMs) on 37 SUTs from prior MT studies and measures MR generation performance.
Does not claim an operator algebraic derivation: this is search/ML based MR generation research (automatic/template style); likely reports effectiveness but not MR provenance, formal boundaries, or operator block coverage.

[52] [Predicting Metamorphic Relation for Matrix Calculation Programs](#)

Karishma Rahman and Upulee Kanewala. ArXiv, 2018. 10 citations.

92% Topic Match

Proposes a machine learning method to predict metamorphic relations for matrix programs.
Uses random walk graph kernels on code/AST graphs to classify/predict applicable MRs, with empirical evaluation on a small function set.
MR identification is treated as automated search/learning; no operator algebra, composition/equivalence laws, formal MR origins, boundary characterizations, or operator block coverage are provided.

[53] [MetBench: A Numerical Expression Metamorphic Relations Benchmark Data Set](#)

Yin Zhao, ..., and Cong Zeng. 2024 11th International Conference on Dependable Systems and Their Applications (DSA), 2024. 1 citations.

92% Topic Match

Proposes a benchmark (MetBench) of numerical-expression metamorphic relations.
Builds a dataset of 139 programs and 924 MRs from open-source, literature, and industry; encodes MRs in LaTeX/SymPy and provides an execution/visualization tool.
Relevant as a taxonomy/benchmark resource for MR cataloging and tooling, but it is not an operator algebraic derivation: MRs are collected/encoded, not systematically generated from operator algebras or given formal origin/boundary proofs.

[54] [Metamorphic Testing of AI-based Applications: A Critical Review](#)

Muhammad Nadeem Khokhar, ..., and Muhammad Fiaz. International Journal of Advanced Computer Science and Applications, 2020. 4 citations.

92% Topic Match

Reviews MT for AI apps, focusing on MR identification and empirical validation.
Surveys prior studies, collects MR examples (e.g., 17 for GAs), and evaluates works using seven empirical criteria (case studies, mutation testing, tools, etc.).
Descriptive/taxonomy style: no operator algebra, no formal composition/equivalence laws, no MR provenance/boundary derivation, nor operator level coverage metrics.

[55] [Using Source Code Metrics for Predicting Metamorphic Relations at Method Level](#)

Alejandra Duque-Torres, ..., and Stefan Fischer. 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), 2022. 9 citations.

91% Topic Match

Proposes an ML-based method to predict applicable metamorphic relations for Java methods. Trains classifiers (SVMs including random-walk-kernel) on 21 source-code features to predict MRs from a predefined list, comparing to CFG-based PMR. MR identification is treated as supervised/template prediction (search/ML); no operator algebra, no formal MR origin/boundary derivation, and no operator-block coverage.

[56] [Metamorphic Relations Identification on Chebyshev Rational Approximation Method in the Nuclide Depletion Calculation Program](#)

Meng Li, ..., and Yaping Wan. 2020 IEEE 20th International Conference on Software Quality, Reliability and Security Companion (QRS-C), 2020. 2 citations.

89% Topic Match

Proposes a domain specific hierarchical MR model (MRHM) and applies MRs to CRAM testing. Builds a three layer MR taxonomy (physics, algorithm, code), derives MRs from burnup equations and CRAM properties, and uses them to test NUIT. MR identification is taxonomy/taxonomy based and expert driven for a specific numerical algorithm; no operator algebraic derivation, formal operator laws, or operator block coverage claims are provided.

[57] [A Replication Study on Predicting Metamorphic Relations at Unit Testing Level](#)

Alejandra Duque-Torres, ..., and Claus Klammer. 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), 2022. 7 citations.

88% Topic Match

Abstract: Metamorphic Testing (MT) addresses the test oracle problem by examining the relations between inputs and outputs of test executions. Such relations are known as Metamorphic Relations (MRs). In current practice, identifying and selecting suitable MRs is usually a challenging manual task, requiring a thorough grasp of the SUT and its application domain. Thus, Kanewala et al. proposed the Predicting Metamorphic Relations (PMR) approach to automatically suggest MRs from a list of six pre-defined MRs for testing newly developed methods. PMR is based on a classification model trained on features extracted from the control-flow graph (CFG) of 100 Java methods. In...

[58] [Metamorphic Testing: A New Approach for Generating Next Test Cases](#)

T. Chen, ..., and S. Yiu. ArXiv, 2020. 649 citations.

87% Topic Match

Abstract: In software testing, a set of test cases is constructed according to some predefined selection criteria. The software is then examined against these test cases. Three interesting observations have been made on the current artifacts of software testing. Firstly, an error-revealing test case is considered useful while a successful test case which does not reveal software errors is usually not further investigated. Whether these successful test cases still contain useful information for revealing software errors has not been properly studied. Secondly, no matter how extensive the testing has been conducted in the development phase, errors may still exist in the...

[59] [Safety-Critical Oracles for Metamorphic Testing of Deep Learning LiDAR Point Cloud Object Detectors](#)

Simon Speth, ..., and Alexander Pretschner. IEEE Open Journal of Intelligent Transportation Systems, 2025. 1 citations.

85% Topic Match

Abstract: Robustness testing is crucial for verifying autonomous vehicles, especially for safety-critical deep learning components like light detection and ranging (LiDAR) object detectors. Metamorphic testing (MT) assesses the robustness by automatically generating test cases based on abstract system specifications known as metamorphic relations (MRs). However, a key challenge is ensuring a traceable safety argumentation for MRs that is in line with industry standards. To ensure this traceability, we derive seven traceable metamorphic transformations from defects identified through interviews with industry experts. Another challenge is prioritizing failures by safety criticality, as not all failing test cases, as evaluated by current intersection over...

[60] [Metamorphic Relation Recognition Method Based on Control Flow Graph Features](#)

Cong Li, ..., and Meng Li. Journal of Physics: Conference Series, 2022. 1 citations.

85% Topic Match

Abstract: Metamorphic testing is an effective method to solve Oracle problems. It is used to test software which is no expected output or the expected output is difficult to construct. However, finding a suitable and effective metamorphic relation in the program is still a very difficult task, usually obtained by domain experts through manual analysis, which greatly reduces the efficiency of software testing. In this work we define a function parameter metamorphic relation from the operation of the function parameter sequence, and preset 10 function parameter metamorphic relation. Because functions with the same structure usually have the same function parameter metamorphic...

[61] [Metamorphic Testing on Nuclide Inventory Tool](#)

Meng Li, ..., and Jie Liu. Unknown journal, 2020. 2 citations.

84% Topic Match

Abstract: The Chebyshev rational approximation method (CRAM) is an important numerical solution algorithm for the burnup equation. Since high complexity of nuclide depletion calculation, its outputs are almost impossible to predict accurately. Therefore, the traditional testing methods are inapplicable, even invalid. The metamorphic testing (MT) is one of promising method to solve such typical testing oracle problem. However, the absent meta-morphic relations (MRs) seriously hinders its application. In response to the nuclear software development process, we established a nuclear MR hierarchical model (NMHM). From top to bottom, it divides MRs into three layers: physics, algorithm and code. From the burnup...

[62] [LLMs in Debate: Does Arguing Make Them Better at Detecting Metamorphic Relations?](#)

Dibyendu Brinto Bose, ..., and Chris Brown. 2025 40th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW), 2025. 2 citations.

81% Topic Match

Abstract: Large Language Models (LLMs) are transforming software engineering, including mobile Augmented Reality (AR) applications. AR software behavior often depends on dynamic environmental factors, making it difficult to use conventional testing and verification approaches. Metamorphic Testing (MT) offers an alternative by assessing whether expected transformations hold across varied conditions. However, there is limited work exploring how well LLMs can detect these transformations-Metamorphic Relations (MRs)-in applications. We propose a stability-driven evaluation framework that examines whether LLMs consistently apply MRs across rephrasings. Our study finds that StarCoder and CodeLlama exhibit higher stability in MR identification compared to the general-purpose model Gemma. Additionally, we...

[63] [MRTCNN: A Lightweight Approach for Predicting Metamorphic Relations](#)

Bo Yang, ..., and Huai Liu. 2024 31st Asia-Pacific Software Engineering Conference (APSEC), 2024. 0 citations.

79% Topic Match

Abstract: Metamorphic testing is a software testing technique that provides both a test case generation strategy and a test result verification mechanism. Its foundation is a set of metamorphic relations, which are basically the necessary properties of the software under test, represented in the form of relationships among multiple inputs and corresponding expected outputs. As the core element of metamorphic testing, metamorphic relations have attracted lots of research interests from different perspectives, among which one major direction is to identify metamorphic relations suitable for certain types of programs. In order to reduce the manual work in the identification process, machine learning...

[64] [Metamorphic Testing: Testing the Untestable](#)

Sergio Segura, ..., and T. Chen. IEEE Software, 2020. 109 citations.

77% Topic Match

Abstract: What if we could know that a program is buggy, even if we could not tell whether or not its observed output is correct? Metamorphic testing provides this ability. This article explains the basics of the technique.

[65] [Hierarchical Metamorphic Relations for Testing Scientific Software](#)

Xuanyi Lin, ..., and Nan Niu. 2018 IEEE/ACM 13th International Workshop on Software Engineering for Science (SE4Science), 2018. 22 citations.

77% Topic Match

Abstract: Scientist developers have not yet routinely adopted systematic testing techniques to assure software quality. A key challenge is the oracle problem, a situation in which appropriate mechanisms are unavailable for checking if the code produces the expected output when executed using a set of test cases (TCs). Metamorphic testing alleviates the oracle problem by specifying the relationship that a source TC and its follow-up TC shall meet. Such relationships are called metamorphic relations (MRs) which are necessary properties of the intended program's functionality. Existing approaches handle the MRs in a flat manner. This paper introduces a novel way to facilitate...

[66] [Measuring Effectiveness of Metamorphic Relations for Image Processing Using Mutation Testing](#)

Fakeeha Jafari and Aamer Nadeem. Journal of Imaging, 2024. 1 citations.

76% Topic Match

Abstract: Testing an intricate plexus of advanced software system architecture is quite challenging due to the absence of test oracle. Metamorphic testing is a popular technique to alleviate the test oracle problem. The effectiveness of metamorphic testing is dependent on metamorphic relations (MRs). MRs represent the essential properties of the system under test and are evaluated by their fault detection rates. The existing techniques for the evaluation of MRs are not comprehensive, as very few mutation operators are used to generate very few mutants. In this research, we have proposed six new MRs for dilation and erosion operations. The fault detection...

[67] [Boosting Metamorphic Relation Prediction via Code Representation Learning: An Empirical Study](#)

Xuedan Zheng, ..., and Zhi Quan Zhou. Software Testing, 2024. 2 citations.

74% Topic Match

Abstract: Metamorphic testing (MT) is an effective testing technique having a broad range of applications. One key task for MT is the identification of metamorphic relations (MRs), which is a fundamental mechanism in MT and is critical to the automation of MT. Prior studies have proposed approaches for predicting MRs (PMR). One major idea behind these PMR approaches is to represent program source code information via manually designed code features and then to apply machine learning-based classifiers to automatically predict whether a specific MR can be applied on the target program. Nevertheless, the human involved procedure of selecting and extracting code features is...

[68] [MetJam: Metamorphic Testing for Data Synthesize and Quality Assurance of an ML-based Jamming \(DOS\) Detector in 5G](#)

N. B. Chaleshtari, ..., and Andrew Walenstein. 2025 IEEE Canadian Conference on Electrical and Computer Engineering (CCECE), 2025. 0 citations.

69% Topic Match

Abstract: Traditionally, metamorphic testing is an approach for quality assurance engineers to generate test cases with oracles for purposes of assuring the functional correctness of a system under test. We provide an argument for utilizing the metamorphic testing technique for the purpose of domain and application specific data synthesis for machine-learning systems, and we propose a process for creating and maintaining a library of metamorphic relations that concisely captures subject matter expert knowledge about classification boundaries in a way that is easy for them to understand and manage. As is typical in machine-learning applications, synthesized ground truth data of the sort...

[69] [Testing Scientific Software: A Systematic Literature Review](#)

Upulee Kanewala and J. Bieman. Information and software technology, 2014. 149 citations.

69% Topic Match

Abstract: Context Scientific software plays an important role in critical decision making, for example making weather predictions based on climate models, and computation of evidence for research publications. Recently, scientists have had to retract publications due to errors caused by software faults. Systematic testing can identify such faults in code. Objective This study aims to identify specific challenges, proposed solutions, and unsolved problems faced when testing scientific software. Method We conducted a systematic literature survey to identify and analyze relevant literature. We identified 62 studies that provided relevant information about testing scientific software. Results We found that challenges faced when testing...

[70] [The integration of machine learning into automated test generation: A systematic mapping study](#)

Afonso Fontes and Gregory Gay. Software Testing, 2022. 20 citations.

68% Topic Match

Abstract: Machine learning (ML) may enable effective automated test generation. We characterize emerging research, examining testing practices, researcher goals, ML techniques applied, evaluation, and challenges in this intersection by performing. We perform a systematic mapping study on a sample of 124 publications. ML generates input for system, GUI, unit, performance, and combinatorial testing or improves the performance of existing generation methods. ML is also used to generate test verdicts, property based, and expected output oracles. Supervised learning—often based on neural networks—and reinforcement learning—often based on Q learning—are common, and some publications also employ unsupervised or semi supervised learning. (Semi /Un)Supervised approaches are evaluated using both...

[71] [A Review of Metamorphic Relation Representation Forms](#)

Yin Zhao, ..., and Jie-Sheng Liu. 2023 10th International Conference on Dependable Systems and Their Applications (DSA), 2023. 3 citations.

67% Topic Match

Abstract: Metamorphic testing can effectively alleviate the problem of testing oracle. The lack of specification descriptions of metamorphic relations affects its generalized application. This text categorizes metamorphic relations into three types based on their representation forms: arithmetic expressions, predicate expressions, and other categories. Among the one hundred and sixty-two relations classified, arithmetic expressions account for sixty-four percent. Within geometry, numerical programs, and algorithms, the main to employ relations on arithmetic expressions, while machine learning and optimization algorithms are mainly inclined toward hiring predicate expression relations. Meanwhile, the research findings about the construction methods of metamorphic relations, the generation of original test...

[72] [Towards a Complete Metamorphic Testing Pipeline](#)

Alejandra Duque-Torres and Dietmar Pfahl. 2023 IEEE International Conference on Software Maintenance and Evolution (ICSME), 2023. 3 citations.

65% Topic Match

Abstract: Metamorphic Testing (MT) addresses the test oracle problem by examining the relationships between input-output pairs in consecutive executions of the System Under Test (SUT). These relations, known as Metamorphic Relations (MRs), specify the expected output changes resulting from specific input changes. However, achieving full automation in generating, selecting, and understanding MR violations poses challenges. Our research aims to develop methods and tools that assist testers in generating MRs, defining constraints, and providing explainability for MR outcomes. In the MR generation phase, we explore automated techniques that utilize a domain-specific language to generate and describe MRs. The MR constraint definition focuses...

[73] [New visions on metamorphic testing after a quarter of a century of inception](#)

T. Chen and T. H. Tse. Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2021. 23 citations.

65% Topic Match

Abstract: Metamorphic testing (MT) was introduced about a quarter of a century ago. It is increasingly being accepted by researchers and the industry as a useful testing technique. The studies, research results, applications, and extensions of MT have given us many insights and visions for its future. Our visions include: MRs will be a practical means to top up test case generation techniques, beyond the alleviation of the test oracle problem; MT will not only be a standalone technique, but conveniently integrated with other methods; MT and MRs will evolve beyond software testing, or even beyond verification; MRs may be anything...

[74] [A New Method for Constructing Metamorphic Relations](#)

Huai Liu, ..., and T. Chen. 2012 12th International Conference on Quality Software, 2012. 87 citations.

63% Topic Match

No summary or abstract available

[75] [Calculation method of metamorphic relational complexity for numerical computation programs based on scale complexity](#)

Angang Ma, ..., and Xiaohua Yang. Unknown journal, 2023. 0 citations.

59% Topic Match

Abstract: Metamorphic testing is an effective way to alleviate the problems of testing Oracle, for which metamorphic relations are key. Studies have shown that the complexity of metamorphic expressions is related to their error detection capability. Therefore, an effective method for calculating the complexity of metamorphic relations can improve the efficiency of metamorphic testing to a certain extent. To address this issue, we analyse the types of input and output components of metamorphic relations described by expressions used in numerical computation programs, define the complexity of metamorphic relations according to the scale complexity in the classification of complexity concepts, and give...

[76] [Metamorphic Testing of Vision-Language Action-Enabled Robots](#)

Pablo Valle, ..., and Aitor Arrieta. ArXiv, 2026. 1 citations.

58% Topic Match

Abstract: Vision-Language-Action (VLA) models are multimodal robotic task controllers that, given an instruction and visual inputs, produce a sequence of low-level control actions (or motor commands) enabling a robot to execute the requested task in the physical environment. These systems face the test oracle problem from multiple perspectives. On the one hand, a test oracle must be defined for each instruction prompt, which is a complex and non-generalizable approach. On the other hand, current state-of-the-art oracles typically capture symbolic representations of the world (e.g., robot and object states), enabling the correctness evaluation of a task, but fail to assess other critical...

[77] [Bug or not Bug? Analysing the Reasons Behind Metamorphic Relation Violations](#)

Alejandra Duque-Torres, ..., and Stefan Fischer. 2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), 2023. 9 citations.

55% Topic Match

Abstract: Metamorphic Testing (MT) is a testing technique that can effectively alleviate the oracle problem. MT uses Metamorphic Relations (MRs) to determine if a test case passes or fails. MRs specify how the outputs should vary in response to specific input changes when executing the System Under Test (SUT). If a particular MR is violated for at least one test input (and its change), there is a high probability that the SUT has a fault. On the other hand, if a particular MR is not violated, it does not guarantee that the SUT is fault free. However, deciding if the MR...

[78] [Metamorphic testing and beyond](#)

T. Chen, ..., and Zhi Quan Zhou. Eleventh Annual International Workshop on Software Technology and Engineering Practice, 2003. 76 citations.

51% Topic Match

Abstract: When testing a program, correctly executed test cases are seldom explored further, even though they may carry useful information. Metamorphic testing proposes to generate follow-up test cases to check important properties of the target function. It does not need a human oracle for output prediction and comparison. In this paper, we highlight the basic concepts of metamorphic testing and some interesting extensions in the areas of program testing, proving, and debugging. Future research directions are also proposed.

[79] [A Template-Based Approach to Describing Metamorphic Relations](#)

Sergio Segura, ..., and Antonio Ruiz-Cortés. 2017 IEEE/ACM 2nd International Workshop on Metamorphic Testing (MET), 2017. 26 citations.

49% Topic Match

Abstract: Metamorphic testing enables the generation of test cases in the absence of an oracle by exploiting relations among different executions of the program under test, called metamorphic relations. In a recent survey, we observed a great variability in the way metamorphic relations are described, typically in an informal manner using natural language. We noticed that the lack of a standard mechanism to describe metamorphic relations often makes them hard to read and understand, which hinders the widespread adoption of the technique. In this paper, we propose a template-based approach for the description of metamorphic relations. The proposed template aims to...

[80] [Composite metamorphic relations for integration testing](#)

Sofia F. Yakusheva and A. Khritankov. Proceedings of the 2022 8th International Conference on Computer Technology Applications, 2022. 1 citations.

49% Topic Match

Abstract: Metamorphic testing is a testing method for problems without test oracles. Integration testing allows for detecting errors in complex systems that may not be found during the testing of their components. In this paper, we propose a novel approach that applies metamorphic testing in integration testing. The main idea is to develop a composite metamorphic relation for the system represented as an acyclic graph. This relation is a logical function of metamorphic relations for the parts of the system (vertices of the graph). It takes into account the features of the parts. Also, it can simplify the search for failure...

[81] [Techniques for Automatic Detection of Metamorphic Relations](#)

Upulee Kanewala. 2014 IEEE Seventh International Conference on Software Testing, Verification and Validation Workshops, 2014. 26 citations.

47% Topic Match

No summary or abstract available

[82] [Metamorphic Testing for Sequential Prediction Models: A Survey of LSTMs and LLMs](#)

Alejandra Duque-Torres, ..., and Claus Klammer. 2026 IEEE International Conference on Software Analysis, Evolution and Reengineering - Companion (SANER-C), 2026. 0 citations.

47% Topic Match

Abstract: Recurrent neural networks such as Long ShortTerm Memory (LSTMs) and Transformer-based Large Language Models (LLMs) are increasingly deployed in applications that rely on sequential predictions, including time-series forecasting, code generation, dialogue systems, and decision support. Ensuring their reliability, robustness, and fairness remains challenging due to the oracle problem, non-determinism, distribution shift, and rapid model evolution. Metamorphic Testing (MT) has emerged as a promising strategy to alleviate the oracle problem by specifying expected relations between multiple executions rather than absolute outputs. However, while a growing effort applies MT to LSTMs and LLMs, the landscape is fragmented across domains, targeted properties, and...

[83] [The Metamorphic Lighthouse: Understanding the Input Data Space of Metamorphic Relations](#)

Alejandra Duque-Torres, ..., and Rudolf Ramler. 2024 50th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), 2024. 1 citations.

43% Topic Match

Abstract: Metamorphic Testing (MT) addresses the test oracle problem by defining how program outputs should change in response to specific input changes. The relations between input changes and their corresponding output changes are called Metamorphic Relations (MRs). Generating suitable MRs is complex and often requires deep domain knowledge. Our previous work introduced MetaTrimmer, a test-data-driven approach for selecting and constraining MRs, involving three steps: Test Data (TD) Generation, MT Process, and MR Analysis. MR Analysis is done to decide whether the violation of an MR for a specific input data pair (original and changed) indicates a failure or simply means that...

[84] [Validation of Dynamic Simulation Models using Metamorphic Testing and Given-When-Then Patterns](#)

Gaadha Sudheerbabu, ..., and K. Klemets. Linköping Electronic Conference Proceedings, 2025. 2 citations.

42% Topic Match

Abstract: As the maritime industry evolves, there is a focus on simulation-driven design, testing, and validation using novel technology solutions. Simulation models designed to represent the behaviour and features of real systems are increasingly available for testing during the early phase of the full development, but in many cases, their testing suffers from the availability of test oracles. Metamorphic testing has become increasingly used in different application domains as an approach to test systems when an explicit test oracle is unavailable. In order to increase its adoption by domain experts, we combine metamorphic testing with Behaviour Driven development for the verification...

[85] [Test Adequacy Criteria for Metamorphic Testing](#)

Yanwen Liu, ..., and Zheng Zheng. 2024 IEEE 24th International Conference on Software Quality, Reliability, and Security Companion (QRS-C), 2024. 1 citations.

38% Topic Match

Abstract: The Oracle problem poses a fundamental challenge in software testing, seeking to validate the accuracy of a program's output given specific test inputs. Metamorphic testing (MT) has emerged as an automated technique to tackle the Oracle problem by testing programs against metamorphic relations (MRs) - crucial properties that persist across multiple executions of the target program. An essential aspect of software testing involves defining objective metrics to gauge the

violation criteria. While many studies concentrate on the defect monitoring capability of MRs, the adequacy criteria for MT often lack coverage information regarding MRs. In this paper, we proposed MTcoverage, an...

[86] [An iterative metamorphic testing technique for web services and case studies](#)

Chang-ai Sun, ..., and T. Chen. Int. J. Web Grid Serv., 2020. 12 citations.

37% Topic Match

Abstract: Metamorphic testing (MT) is an innovative approach to alleviating the oracle problem in software testing, which uses metamorphic relations of the program under test, instead of the test oracles, to verify its outputs. To alleviate the oracle problem of testing web services, we had previously proposed an MT framework for web services. In this paper, we further improve the efficiency and automation of this framework by leveraging metamorphic relations to iteratively generate test cases. We present a fixed-size iterative MT algorithm and implement it in the MT framework. We conduct three case studies to evaluate the fault detection effectiveness and...

[87] [Bottom-up Integration Testing with the Technique of Metamorphic Testing](#)

Xiaoyuan Xie, ..., and Baowen Xu. 2014 14th International Conference on Quality Software, 2014. 5 citations.

35% Topic Match

No summary or abstract available

[88] [Minimum Complete MR Subsets under Semantic-Mutation Fault Models: A Support-Set Domination Boundary](#)

Meng Li, ..., and Shiyu Yan. ArXiv, 2026. 0 citations.

34% Topic Match

Abstract: This paper asks when MR-subset selection is a real mutant-level requirement for minimum complete evidence in metamorphic testing rather than a coarse fault-class counting artifact. We define a layer-relative completeness criterion over an admitted mutant--draw coverage universe. The central result is a support-set domination boundary: it states when class-level abstraction is safe and when mutant-level MR minimization is necessary. The boundary is governed by kill-signature heterogeneity, which yields a scoped fault-signature kernel and separates the MR-specific question from ordinary fault-class counting. The resulting Min-MR-Complete problem is Set-Cover-equivalent over the selected coverage universe, giving NP-hardness, the classical logarithmic approximation boundary, a...

[89] [Theoretical and Empirical Analyses of the Effectiveness of Metamorphic Relation Composition](#)

Kun Qiu, ..., and P. Poon. IEEE Transactions on Software Engineering, 2022. 40 citations.

32% Topic Match

Abstract: Metamorphic Relations (MRs) play a key role in determining the fault detection capability of Metamorphic Testing (MT). As human judgement is required for MR identification, systematic MR generation has long been an important research area in MT. Additionally, due to the extra program executions required for follow-up test cases, some concerns have been raised about MT cost-effectiveness. Consequently, the reduction in testing costs associated with MT has become another important issue to be addressed. MR composition can address both of these problems. This technique can automatically generate new MRs by composing existing ones, thereby reducing the number of follow-up test...

[90] [MDPMorph: An MDP-Based Metamorphic Testing Framework for Deep Reinforcement Learning Agents](#)

Jiapeng Li, ..., and Valerio Terragni. 2025 IEEE 36th International Symposium on Software Reliability Engineering (ISSRE), 2025. 1 citations.

32% Topic Match

Abstract: Deep Reinforcement Learning (DRL) systems are widely used across various domains. However, testing these systems presents significant challenges. The DRL agent, which serves as the core decision-maker, generates continuous value estimates rather than discrete labels and operates under nonstationary policies within complex and stochastic environments. Consequently, there is no definitive "correct answer" for each state-action pair, complicating automated test generation due to the known oracle problem. To address this challenge, we propose a Metamorphic Testing (MT) framework (MDPMORPH) specifically designed for validating DRL agents. Our framework is based on Markov Decision Processes (MDP) and focuses on the core reasoning properties...

[91] [Metamorphic Testing for Audio Content Moderation Software](#)

Wenxuan Wang, ..., and Michael R. Lyu. 2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE), 2025. 0 citations.

30% Topic Match

Abstract: The rapid growth of audio-centric platforms and applications such as Whatsapp and Twitter has transformed the way people communicate and share audio content in modern society. However, these platforms are increasingly misused to disseminate harmful audio content, such as hate speech, deceptive advertisements, and explicit material, which can have significant negative consequences (e.g., detrimental effects on mental health). In response, researchers and practitioners have been actively developing and deploying audio content moderation tools to tackle this issue. Despite these efforts, malicious actors can bypass moderation systems by making subtle alterations to audio content, such as modifying pitch or inserting noise...

[92] [Feedback-Directed Metamorphic Testing](#)

Chang-ai Sun, ..., and T. Chen. ACM Transactions on Software Engineering and Methodology, 2022. 18 citations.

30% Topic Match

Abstract: Over the past decade, metamorphic testing has gained rapidly increasing attention from both academia and industry, particularly thanks to its high efficacy on revealing real-life software faults in a wide variety of application domains. On the basis of a set of metamorphic relations among multiple software inputs and their expected outputs, metamorphic testing not only provides a test case generation strategy by constructing new (or follow-up) test cases from some original (or source) test cases, but also a test result verification mechanism through checking the relationship between the outputs of source and follow-up test cases. Many efforts have been made...

[93] [Optimizing Metamorphic Testing: Prioritizing Relations Through Execution Profile Dissimilarity](#)

Madhusudan Srinivasan and Upulee Kanewala. ArXiv, 2024. 1 citations.

29% Topic Match

Abstract: An oracle determines whether the output of a program for executed test cases is correct. For machine learning programs, such an oracle is often unavailable or impractical to apply. Metamorphic testing addresses this by using metamorphic relations (MRs), which are essential properties of the software under test, to verify program correctness. Prioritizing MRs enhances fault detection effectiveness and improves testing efficiency. However, fault-based MR prioritization is costly, and code coverage-based approaches often yield inaccurate results. To address these challenges, we propose a statement centrality-based prioritization approach that leverages diversity in the execution profiles of source and follow-up test cases. Our...

[94] [VeriEQ: Finding Verilog Simulators and Synthesizers Bugs with Equivalence Circuit Transformation](#)

Zhenyu Yan, ..., and Yu Jiang. Proceedings of the ACM on Programming Languages, 2026. 0 citations.

25% Topic Match

Abstract: Verilog simulators and synthesizers play a critical role in chip design and verification. However, due to the complexity of simulation and synthesis processes, they are prone to introducing various types of bugs. Among these, Behavioral Deviation Bugs (BDBs) are particularly severe, as they can cause incorrect results by introducing subtle semantic deviations. Such deviations make the chip behave differently from its intended design and may even enable hardware backdoors. In this work, we propose VeriEQ, an automated framework based on the concept of metamorphic testing, to detect BDBs by generating semantically equivalent Verilog programs. First, to increase the likelihood of...

[95] [MT-PART: Metamorphic-Testing-Based Adaptive Random Testing Through Partitioning](#)

Zhihao Ying, ..., and Zhi Quan Zhou. 2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC), 2024. 4 citations.

25% Topic Match

Abstract: Metamorphic Testing (MT) has been repeatedly proven effective in detecting software faults. MT detects faults by checking the Metamorphic Relations (MRs) among Source Test Cases (STCs) and Follow-up Test Cases (FTCs) and the corresponding outputs. Metamorphic Groups (MGs) denote the associated STCs and FTCs. The performance of MT relates strongly to the MRs and MGs. However, previous studies that on MG generation mainly focused on improving the effectiveness (i.e. fault-detection capability) of MT, but to some extent overlooked the efficiency. This paper proposes a new kind of MG generation algorithms called Metamorphic-Testing-based Adaptive Random Testing through Partitioning (MT-PART). These algorithms...

[96] [Metamorphic testing of large language model question answering systems](#)

Xiaoyin Wang and Xinyu Liu. Unknown journal, 2025. 0 citations.

21% Topic Match

Abstract: Large language models are capable of producing a wide variety of creative outputs, and their question-answering capabilities are compelling. Current testing efforts for large language models question answering systems are largely based on the reference paradigm and rely on manual labeling of test inputs. Testing of Large language models suffers from the test oracle problem. Metamorphic testing can test software by defining the expected relationships between inputs and outputs to ensure that the software still works correctly in different situations. In this paper, we propose a metamorphic testing method for large language models question answering systems. Specifically, starting from the...

[97] [Prioritizing Metamorphic Relations via Execution Profile Dissimilarity for Improved Fault Detection](#)

Madhusudan Srinivasan and Upulee Kanewala. 2025 IEEE International Conference on Artificial Intelligence Testing (AITest), 2025. 0 citations.

21% Topic Match

Abstract: An oracle is a mechanism that determines whether the output of the program for the executed test cases is correct. For machine learning programs, such an oracle is not available or is too difficult to apply. Metamorphic testing is a testing approach that uses metamorphic relations, which are necessary properties of the software under test to help verify the correctness of a program. Prioritization of metamorphic relations helps to increase fault detection effectiveness and improve metamorphic testing efficiency. However, prioritizing metamorphic relations based on faults and code coverage is often not effective for prioritizing MRs since fault-based ordering of MRs...

[98] [Large Language Models: The Next Frontier for Variable Discovery within Metamorphic Testing?](#)

Christos Tsigkanos, ..., and Timo Kehrer. 2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), 2023. 22 citations.

20% Topic Match

Abstract: Metamorphic testing involves reasoning on necessary properties that a program under test should exhibit regarding multiple input and output variables. A general approach consists of extracting metamorphic relations from auxiliary artifacts such as user manuals or documentation, a strategy particularly fitting to testing scientific software. However, such software typically has large input-output spaces, and the fundamental prerequisite – extracting variables of interest – is an arduous and non-scalable process when performed manually. To this end, we devise a workflow around an autoregressive transformer-based Large Language Model (LLM) towards the extraction of variables from user manuals of scientific software. Our end-to-end...

[99] [Metamorphic relation prioritization for effective regression testing](#)

Madhusudan Srinivasan and Upulee Kanewala. Software Testing, 2021. 18 citations.

19% Topic Match

Abstract: Metamorphic testing (MT) is widely used for testing programs that face the oracle problem. It uses a set of metamorphic relations (MRs), which are relations among multiple inputs and their corresponding outputs to determine whether the program under test is faulty. Typically, MRs vary in their ability to detect faults in the program under test, and some MRs tend to detect the same set of faults. In this paper, we propose approaches to prioritize MRs to improve the efficiency and effectiveness of MT for regression testing. We present two MR prioritization approaches: (i) fault based and (ii) coverage based. To evaluate these...

[100] [Metamorphic Relations Prioritization And Selection Based on Test Adequacy Criteria](#)

Dafei Huang, ..., and Meng Li. 2022 4th International Academic Exchange Conference on Science and Technology Innovation (IAECST), 2022. 4 citations.

19% Topic Match

Abstract: Metamorphic testing is the mainstream technique to solve oracle problems, and the effectiveness of metamorphic relations directly affects the testing cost. Most existing studies are summaries of engineering experience and lack theoretical discussions. A distance-based definition of metamorphic relation validity is provided, and a quantitative relationship model based on distance and test coverage is established to guide metamorphic relation selection and ranking. Experiments on four procedures and nearly 50 metamorphic relations show that, compared with random, coverage-based, and distance-based methods, this method has higher defect detection capability and can improve test effectiveness under the condition of using the same number...

[101] [Test It Before You Trust It: Applying Software Testing for Trustworthy In-context Learning](#)

Teeradaj Racharak, ..., and T. Sunetnanta. Unknown journal, 2025. 3 citations.

17% Topic Match

Abstract: In-context learning (ICL) has emerged as a powerful capability of large language models (LLMs), enabling them to perform new tasks based on a few provided examples without explicit fine-tuning. Despite their impressive adaptability, these models remain vulnerable to subtle adversarial perturbations and exhibit unpredictable behavior when faced with linguistic variations. Inspired by software testing principles, we introduce a software testing-inspired framework, called MMT4NL, for evaluating the trustworthiness of in-context learning by utilizing adversarial perturbations and software testing techniques. It includes diverse evaluation aspects of linguistic capabilities for testing the ICL capabilities of LLMs. MMT4NL is built around the idea of...

[102] [MTF: an Open-Source Metamorphic Testing Framework for LLM-based systems](#)

Theis Henry, ..., and Masahide Nakamura. Proceedings of the 2025 5th International Conference on Artificial Intelligence and Application Technologies, 2025. 1 citations.

15% Topic Match

Abstract: Metamorphic testing is an emerging approach to solve oracle problems. It relies on relations among different executions of the System Under Test (SUT), called metamorphic relations. This article presents an open-source metamorphic testing framework dedicated to validating LLM-based web services applications. It is decomposed into a metamorphic test generator and a test driver. Both rely on a predefined set of metamorphic relations implemented as libraries that can be completed.

[103] [Metamorphic Testing and Its Applications](#)

Z. Zhou, ..., and T. Chen. Unknown journal, 2004. 123 citations.

15% Topic Match

No summary or abstract available

[104] [How Composite Metamorphic Relations Enhance Test Effectiveness of DNN Testing: An Empirical Study](#)

Huayao Wu, ..., and Tsong Yueh Chen. IEEE Transactions on Software Engineering, 2026. 0 citations.

15% Topic Match

Abstract: Metamorphic Testing (MT) is a powerful technique to alleviate the test oracle problem of DNN testing. At its core is a set of Metamorphic Relations (MRs), which indicate necessary properties that multiple test inputs and their outputs should satisfy. Recently, researchers have proposed to compose multiple individual MRs to create new Composite Metamorphic Relations (CMRs), in the hope of enhancing the cost-effectiveness of MT. However, the potential benefit of such compositions is yet to be systematically explored for DNN based systems. In this paper, we present the first empirical study that investigates the relative test effectiveness of CMRs over individual...

[105] [SATORI: Static Test Oracle Generation for REST APIs](#)

Juan C. Alonso, ..., and Antonio Ruiz-Cortés. 2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE), 2025. 3 citations.

12% Topic Match

Abstract: REST API test case generation tools are evolving rapidly, with growing capabilities for the automated generation of complex tests. However, despite their strengths in test data generation, these tools are constrained by the types of test oracles they support, often limited to crashes, regressions, and noncompliance with API specifications or design standards. This paper introduces SATORI (Static API Test Oracle Inference), a black-box approach for generating test oracles for REST APIs by analyzing their OpenAPI Specification. SATORI uses large language models to infer the expected behavior of an API by analyzing the properties of the response fields of its operations,...

[106] [Using Metamorphic Relations in Redundancy-based Fault/Intrusion Tolerance](#)

F. Giandomenico, ..., and Antonia Bertolino. ACM Transactions on Software Engineering and Methodology, 2025. 0 citations.

12% Topic Match

Abstract: Redundancy is widely used as a method for fault and intrusion tolerance. However, if the redundant components lack sufficient diversity, potentially dangerous common mode failures may go undetected. To address this issue, the design diversity approach has been proposed in the literature for decades. In this

paper, we take an innovative approach to this problem by introducing a broader notion of diversity, which leverages Metamorphic Relations (MRs), i.e., necessary properties that must hold among diverse inputs and diverse outputs. We define two generic categories of MRs that establish data diversity and functional diversity. Furthermore, we elaborate on two corresponding logical...

[107] [MRGS ART: Metamorphic Relation and Group Selection Based on Adaptive Random Testing](#)

Zhihao Ying, ..., and Zhi Quan Zhou. Software Testing, 2024. 6 citations.

11% Topic Match

Abstract: Metamorphic testing (MT) is effective in detecting software failures; it detects failures by examining the metamorphic relations (MRs) among source test cases (STCs), follow up test cases (FTCs) and their respective outputs. The STCs together with the corresponding FTCs, considered as a whole, are called metamorphic groups (MGs). MT performance relies heavily on the MRs and MGs. Previous studies have mainly focused on improving MT performance by identifying effective MRs, or through generation of MGs with high quality, but have somewhat neglected the selection of MRs and MGs from existing ones. In this paper, we address this issue by introducing a...

[108] [GeMTest: A General Metamorphic Testing Framework](#)

Simon Speth and Alexander Pretschner. 2025 IEEE/ACM 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion), 2025. 3 citations.

10% Topic Match

Abstract: Metamorphic testing (MT) is an established software testing methodology suitable for testing various types of systems under test (SUTs), but identifying and implementing metamorphic relations (MRs) remains a challenge. This paper presents GeMTest, a general-purpose metamorphic testing framework that is domain-independent, enabling software testers to implement MRs in Python and execute them with pytest. In GeMTest, MRs are implemented using custom decorators to annotate Python functions, which specify the follow-up generation function, the metamorphic oracle, and the SUT. GeMTest then automatically creates and executes a pytest test suite containing multiple metamorphic test cases derived from the user-defined MRs. We evaluate...

[109] [Metamorphic Testing of Relation Extraction Models](#)

Yuhe Sun, ..., and Mingyue Jiang. Algorithms, 2023. 4 citations.

10% Topic Match

Abstract: Relation extraction (RE) is a fundamental NLP task that aims to identify relations between some entities regarding a given text. RE forms the basis for many advanced NLP tasks, such as question answering and text summarization, and thus its quality is critical to the relevant downstream applications. However, evaluating the quality of RE models is non-trivial. On the one hand, obtaining ground truth labels for individual test inputs is tedious and even difficult. On the other hand, there is an increasing need to understand the characteristics of RE models in terms of various aspects. To mitigate these issues, this study...

[110] [Selecting and Constraining Metamorphic Relations](#)

Alejandra Duque-Torres. 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion), 2024. 3 citations.

9% Topic Match

Abstract: Software testing is a critical aspect of ensuring the reliability and quality of software systems. However, it often poses challenges, particularly in determining the expected output of a System Under Test (SUT) for a given set of inputs, a problem commonly referred to as the test oracle problem. Metamorphic Testing (MT) offers a promising solution to the test oracle problem by examining the relations between input-output pairs in consecutive executions of the SUT. These relations, referred to as Metamorphic Relations (MRs), define the expected changes in the output when specific changes are made to the input. Our research is focused...

[111] [Using Metamorphic Relations to Improve The Effectiveness of Automatically Generated Test Cases](#)

Prashanta Saha and Upulee Kanewala. 2022 IEEE/ACIS 20th International Conference on Software Engineering Research, Management and Applications (SERA), 2022. 0 citations.

9% Topic Match

Abstract: Automated test case generation has helped to reduce the cost of testing. However, developing effective test oracles for these automatically generated test cases still remains a challenge. Metamorphic testing (MT) has become a well-known software testing approach over the years. This testing technique can effectively alleviate the oracle problem faced when testing using metamorphic relations (MRs) to determine whether a test case is passed or failed. In this work, we conduct an empirical study on an open source linear algebra library to evaluate whether MRs can be utilized to improve the fault detection effectiveness of automatically generated test cases. Our...

[112] [OneMoreTest: A Learning-Based Approach to Generating and Selecting Fault-Revealing Unit Tests](#)

Wei Wei, ..., and Hui Liu. IEEE Transactions on Software Engineering, 2025. 0 citations.

8% Topic Match

Abstract: Developers often manually design a few unit tests for a given method under development. After passing such manually designed tests, however, they usually have to turn to automated test case generation tools like EvoSuite and Randoop for more thorough testing. Although the automatically generated tests may achieve a high coverage, they rarely identify hard-to-detect defects automatically because of the well-known test oracle problem: It is challenging to tell whether the output is correct or incorrect without explicit test oracle (expected output). Consequently, developers should manually select and verify a few suspicious test cases to identify hard-to-detect defects. To this end...

[113] [JFuzz: A Tool for Automated Java Unit Testing Based on Data Mutation and Metamorphic Testing Methods](#)

Hong Zhu. 2015 Second International Conference on Trustworthy Systems and Their Applications, 2015. 42 citations.

7% Topic Match

No summary or abstract available

[114] [Prioritization of Metamorphic Relations Based on Test Case Execution Properties](#)

Madhusudan Srinivasan. 2018 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW), 2018. 7 citations.

6% Topic Match

Abstract: A test oracle is essential for software testing. In certain complex systems, it is hard to distinguish between correct and incorrect behavior. Metamorphic testing is one of the solution to solve the test oracle problem. In metamorphic testing, metamorphic relations (MRs) are derived based on the properties exhibited by the program under test (PUT). These MRs play a major role in the generation of test data for conducting MT. The effectiveness of MRs can be determined based on the ability to detect considerable faults for the given PUT. Many metamorphic relations with different fault finding capability can be used to...

[115] [An Approach to Testing Banking Software Using Metamorphic Relations](#)

Karishma Rahman and C. Izurieta. 2023 IEEE 24th International Conference on Information Reuse and Integration for Data Science (IRI), 2023. 1 citations.

6% Topic Match

Abstract: Software systems used for banking are crucial for daily operations and are considered to be part of critical infrastructure; however, testing the functions of these highly reusable systems can be difficult due to the project's complexity and the absence of a reliable oracle. In software testing, the Oracle problem directs to the difficulty of deciding whether the software's observed behavior is correct. To address this issue, we suggest utilizing metamorphic testing (MT), which tests the banking system's functionalities based on their properties. Metamorphic testing is a software testing technique where multiple inputs are generated for a program, then those inputs...

[116] [Augmented example-based synthesis using relational perturbation properties](#)

Shengwei An, ..., and R. Samanta. Proceedings of the ACM on Programming Languages, 2019. 16 citations.

6% Topic Match

Abstract: Example-based specifications for program synthesis are inherently ambiguous and may cause synthesizers to generate programs that do not exhibit intended behavior on unseen inputs. Existing synthesis techniques attempt to address this problem by either placing a domain-specific syntactic bias on the hypothesis space or heavily relying on user feedback to help resolve ambiguity. We present a new framework to address the ambiguity/generalizability problem in example-based synthesis. The key feature of our framework is that it places a semantic bias on the hypothesis space using "relational perturbation properties" that relate the perturbation/change in a program output to the perturbation/change in a...

[117] [Identifying the Failure-Revealing Test Cases in Metamorphic Testing: A Statistical Approach](#)

Zheng Zheng, ..., and Tiancheng Li. ACM Transactions on Software Engineering and Methodology, 2024. 4 citations.

5% Topic Match

Abstract: Metamorphic testing, thanks to its high failure-detection effectiveness especially in the absence of test oracle, has been widely applied in both the traditional context of software testing and other relevant fields such as fault localization and program repair. Its core element is a set of metamorphic relations, which are the necessary properties of the target algorithm in the form of the relationships among multiple inputs and corresponding expected outputs. When a relation is violated by the outputs of a group of test cases, namely metamorphic group of test cases, that are constructed based on the relation, a failure is said...

[118] [An Empirical Study on Common Sense-Violating Bugs in Mobile Apps](#)

Fu Fan, ..., and Hui Liu. ACM Transactions on Software Engineering and Methodology, 2024. 1 citations.

5% Topic Match

Abstract: Mobile applications are widely used by billions of users in their daily work and life. Such GUI software is prone to bugs, potentially degrading user experience. Notably, many bugs in mobile apps are reported by end-users who cannot access the requirements of the app or test cases accompanied by explicitly specified test oracles. It may suggest that such bugs are not identified in the traditional way, i.e., by comparing the actual behaviors of the apps against their expected behaviors explicitly specified in the requirements or test cases. Instead, such bugs are often identified by comparing the actual behaviors against users'...

[119] [A Tale from the Trenches: Applying Metamorphic and Differential Testing to Bioinformatics Software](#)

Alexis L. Marsh, ..., and R. Cottingham. 2025 IEEE Conference on Software Testing, Verification and Validation (ICST), 2025. 0 citations.

4% Topic Match

Abstract: Metamorphic and differential testing have been proposed as best practices for testing software that is difficult to test, such as for programs in scientific domains. An assumption is that these approaches can be easily customized and applied to almost any domain. However, scientific software is often data-driven, and metamorphic relations may require significant domain knowledge to develop. In addition, tools are often written for ad-hoc experimentation by the scientists and often embed many assumptions about the importance and representation of different natural phenomena. In this paper, we present our experience applying both metamorphic and differential testing to a set of...

[120] [Ontology of Mutation Testing for Java Operators](#)

Sherolwendy Anak Sualim, ..., and N. Saadon. International Journal of Innovative Computing, 2018. 3 citations.

4% Topic Match

Abstract: Operators are special characters within the Java language to manipulate primitive data type. Java operators can be classified as unary, binary and ternary. The design of Java operator sometimes becomes confusing when it comes to testing tools as they had the same function with different label in every testing tool. Therefore, in order to map the knowledge of operators correctly, this research has proposed ontology that is dedicated to mutation testing as a means to define the formal specification of concepts and documentation of knowledge of Java operators. Existing papers on ontology did not specify further on entities and properties...

[121] [Performance Metamorphic Testing: Motivation and Challenges](#)

Sergio Segura, ..., and Antonio Ruiz-Cortés. 2017 IEEE/ACM 39th International Conference on Software Engineering: New Ideas and Emerging Technologies Results Track (ICSE-NIER), 2017. 29 citations.

4% Topic Match

Abstract: Performance testing is a challenging task mainly due to the lack of test oracles, that is, mechanisms to decide whether the performance of a program under a certain workload is either acceptable or poor due to a performance bug. Metamorphic testing enables the generation of test cases in the absence of an oracle by exploiting the relations (so-called metamorphic relations) between the inputs and outputs of multiple executions of the program under test. In the last two decades, metamorphic testing has been successfully used to detect functional faults in a variety of domains, ranging from web services to simulators. However,...

[122] [On the Correlation between the Effectiveness of Metamorphic Relations and Dissimilarities of Test Case Executions](#)

Yuxiang Cao, ..., and T. Chen. 2013 13th International Conference on Quality Software, 2013. 49 citations.

4% Topic Match

Abstract: Metamorphic testing (MT) is a property-based automated software testing method. It alleviates the oracle problem by testing programs against metamorphic relations (MRs), which are necessary properties among multiple executions of the target program. For a given problem, usually more than one MR can be identified. It is therefore of practical importance for testers to know the nature of good MRs, that is, which MRs are likely to have higher chances of revealing failures. To address this issue we investigate the correlation between the fault-detection effectiveness of MRs and the dissimilarity (distance) of test case execution profiles. Empirical study results reveal...

[123] [Self-Checked Metamorphic Testing of an Image Processing Program](#)

Junhua Ding, ..., and Xin-Hua Hu. 2010 Fourth International Conference on Secure Software Integration and Reliability Improvement, 2010. 28 citations.

3% Topic Match

No summary or abstract available

[124] [A Machine Learning Approach for Developing Test Oracles for Testing Scientific Software](#)

Junhua Ding and Dongmei Zhang. Unknown journal, 2016. 10 citations.

3% Topic Match

No summary or abstract available

[125] [Intramorphic Testing: A New Approach to the Test Oracle Problem](#)

Manuel Rigger and Z. Su. Proceedings of the 2022 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software, 2022. 11 citations.

3% Topic Match

Abstract: A test oracle determines whether a system behaves correctly for a given input. Automatic testing techniques rely on an automated test oracle to test the system without user interaction. Important families of automated test oracles include Differential Testing and Metamorphic Testing, which are both black-box approaches; that is, they provide a test oracle that is oblivious to the system's internals. In this work, we propose Intramorphic Testing as a white-box methodology to tackle the test oracle problem. To realize an Intramorphic Testing approach, a modified version of the system is created, for which, given a single input, a test oracle...

[126] [Research on Metamorphic Testing: A Case Study in Integer Bugs Detection](#)

Yao Yi, ..., and Zhengping Ren. 2013 Fourth International Conference on Intelligent Systems Design and Engineering Applications, 2013. 5 citations.

2% Topic Match

Abstract: The integer errors are harmful for the security of software. Because of the Test Oracle problem, integer bugs are always ignored. Metamorphic Testing can prove a way to determine the correctness of testing outputs with metamorphic relations between a series of outputs that correspond to a series of inputs, based on necessary properties of software under testing. In this paper, a metamorphic relationship is proposed which can detect invisible integer bugs without oracle. It is shown in our case study that this method can detect some invisible errors which are difficult to be found in conventional approach and improve the...

[127] [Application of Property-based Testing Tools for Metamorphic Testing](#)

Nasser Alzahrani, ..., and James Harland. ArXiv, 2022. 2 citations.

2% Topic Match

Abstract: Metamorphic testing (MT) is a general approach for the testing of a specific kind of software systems -- so-called "non-testable", where the "classical" testing approaches are difficult to apply. MT is an effective approach for addressing the test oracle problem and test case generation problem. The test oracle problem is when it is difficult to determine the correct expected output of a particular test case or to determine whether the actual outputs agree with the expected outcomes. The core concept in MT is metamorphic relations (MRs) which provide formal specification of the system under test. One of the challenges in...

[128] [Performance metamorphic testing: A Proof of concept](#)

Sergio Segura, ..., and Antonio Ruiz-Cortés. Inf. Softw. Technol., 2018. 18 citations.

2% Topic Match

Abstract: Abstract Context Performance testing is a challenging task mainly due to the lack of test oracles, i.e. mechanisms to decide whether the performance of a program is acceptable or not because of a bug. Metamorphic testing enables the generation of test cases in the absence of an oracle by exploiting the so-called metamorphic relations between the inputs and outputs of multiple executions of the program under test. In the last two decades, metamorphic testing has been successfully used to detect functional faults in different domains. However, its applicability to performance testing remains unexplored. Objective We propose the application of metamorphic...

[129] [MR-Coupler: Automated Metamorphic Test Generation via Functional Coupling Analysis](#)

Congying Xu, ..., and S. Cheung. ArXiv, 2026. 1 citations.

2% Topic Match

Abstract: Metamorphic testing (MT) is a widely recognized technique for alleviating the oracle problem in software testing. However, its adoption is hindered by the difficulty of constructing effective metamorphic relations (MRs), which often require domain-specific or hard-to-obtain knowledge. In this work, we propose a novel approach that leverages the functional coupling between methods, which is readily available in source code, to automatically construct MRs and generate metamorphic test cases (MTCs). Our technique, MR-Coupler, identifies functionally coupled method pairs, employs large language models to generate candidate MTCs, and validates them through test amplification and mutation analysis. In particular, we leverage three functional...

[130] [Software Testing as A Problem of Machine Learning -- A Survey and Critical Review](#)

Hong Zhu. Journal Not Provided, 2018. 0 citations.

1% Topic Match

No summary or abstract available

[131] [Engineering for Computational Science : Past , Present , Future](#)

A. Johanson and W. Hasselbring. Journal Not Provided, 2019. 0 citations.

1% Topic Match

No summary or abstract available

[132] [A Test Suite Minimization Technique for Testing Numerical Programs](#)

Prashanta Saha, ..., and Upulee Kanewala. 2023 IEEE/ACIS 21st International Conference on Software Engineering Research, Management and Applications (SERA), 2023. 0 citations.

1% Topic Match

Abstract: Metamorphic testing is a technique that uses metamorphic relations (i.e., necessary properties of the software under test), to construct new test cases (i.e., follow-up test cases), from existing test cases (i.e., source test cases). Metamorphic testing allows for the verification of testing results without the need of test oracles (a mechanism to detect the correctness of the outcomes of a program), and it has been widely used in many application domains to detect real-world faults. Numerous investigations have been conducted to further improve the effectiveness of metamorphic testing. Recent studies have emerged suggesting a new research direction on the generation...

[133] [Using Metamorphic Relation Violation Regions to Support a Simulation Framework for the Process of Metamorphic Testing](#)

Zhihao Ying, ..., and Z. Zhou. 2022 IEEE 46th Annual Computers, Software, and Applications Conference (COMPSAC), 2022. 6 citations.

1% Topic Match

Abstract: Metamorphic testing (MT) has been growing in popularity, but it can still be quite challenging and time-consuming to assess its performance. Typical approaches to performance assessment can require a series of steps, and depend on a variety of factors, often requiring serendipity. This can be a bottleneck for some aspects of MT research. Central to MT, metamorphic relations (MRs) represent necessary properties of the system under test (SUT). In traditional software testing, simulations are often employed to examine and compare the performance of different testing strategies. However, these simulations are typically designed based on the assumed availability (and applicability) of...

[134] [Retromorphic Testing with Hierarchical Verification for Hallucination Detection in RAG](#)

Boxi Yu, ..., and Emir Muñoz. ArXiv, 2026. 0 citations.

1% Topic Match

Abstract: Large language models (LLMs) continue to hallucinate in retrieval-augmented generation (RAG), producing claims that are unsupported by or conflict with the retrieved context. Detecting such errors remains challenging when faithfulness is evaluated solely with respect to the retrieved context. Existing approaches either provide coarse-grained, answer-level scores or focus on open-domain factuality, often lacking fine-grained, evidence-grounded diagnostics. We present RT4CHART, a retromorphic testing framework for context-faithfulness assessment. RT4CHART decomposes model outputs into independently verifiable claims and performs hierarchical, local-to-global verification against the retrieved context. Each claim is assigned one of three labels: entailed, contradicted, or baseless. Furthermore, RT4CHART maps claim-level decisions...

[135] [Testing Causality in Scientific Modelling Software](#)

A. Clark. Journal Not Provided, 2023. 0 citations.

1% Topic Match

No summary or abstract available

[136] [Fault Detection Effectiveness of Source Test Case Generation Strategies for Metamorphic Testing](#)

Prashanta Saha and Upulee Kanewala. 2018 IEEE/ACM 3rd International Workshop on Metamorphic Testing (MET), 2018. 16 citations.

1% Topic Match

Abstract: Metamorphic testing is a well known approach to tackle the oracle problem in software testing. This technique requires the use of source test cases that serve as seeds for the generation of follow-up test cases. Systematic design of test cases is crucial for the test quality. Thus, source test case generation strategy can make a big impact on the fault detection effectiveness of metamorphic testing. Most of the previous studies on metamorphic testing have used either random test data or existing test cases as source test cases. There has been limited research done on systematic source test case generation for...

[137] [Learning Correct Behavior from Examples: Validating Sequential Execution in Autonomous Agents](#)

Reshabh K Sharma, ..., and Yu Hu. ArXiv, 2026. 0 citations.

1% Topic Match

Abstract: As autonomous agents become increasingly sophisticated, validating their sequential behavior presents a significant challenge. Traditional testing approaches require manual specification, exact sequence matching, or thousands of training examples. We present a novel algorithm that automatically learns correct behavior from just 2-10 passing execution traces and validates new executions against this learned model. Our approach combines dominator analysis from compiler theory with multimodal large language model-powered semantic understanding to identify essential states and handle non-deterministic behavior. The system constructs a generalized ground truth model using Prefix Tree Acceptors, merges traces through multi-tiered equivalence detection, and validates new executions via topological subsequence...

[138] [Path-directed source test case generation and prioritization in metamorphic testing](#)

Chang-ai Sun, ..., and Huai Liu. J. Syst. Softw., 2022. 31 citations.

0% Topic Match

Abstract: Abstract Metamorphic testing is a technique that makes use of some necessary properties of the software under test, termed as metamorphic relations, to construct new test cases, namely follow-up test cases, based on some existing test cases, namely source test cases. Due to the ability of verifying testing results without the need of test oracles, it has been widely used in many application domains and detected lots of real-life faults. Numerous investigations have been conducted to further improve the effectiveness of metamorphic testing, most of which were focused on the identification and selection of "good" metamorphic relations. Recently, a few...

[139] [Search-based synthesis of equivalent method sequences](#)

A. Goffi, ..., and P. Tonella. Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, 2014. 36 citations.

0% Topic Match

No summary or abstract available

[140] [Applying metamorphic testing to reliability calculating programs](#)

Taito Asaji and Tatsuhiro Tsuchiya. 2023 IEEE 28th Pacific Rim International Symposium on Dependable Computing (PRDC), 2023. 0 citations.
0% Topic Match

Abstract: Reliability evaluation is imperative for developing safety-critical systems where failures can cause serious consequences. Thus, assuring the correctness of programs that compute reliability is indispensable. However, this is a challenging task when the correct output is not known a priori, since it is impossible to determine whether the calculated reliability is correct or not. To overcome this challenge, we propose the application of metamorphic testing to reliability calculating programs. Metamorphic testing tests software by checking whether particular relations, generally called metamorphic relations, hold or not between multiple inputs and outputs rather than solely focusing on the correctness of independent outputs....

[141] [The Oracle Problem in Software Testing: A Survey](#)

Earl T. Barr, ..., and S. Yoo. IEEE Transactions on Software Engineering, 2015. 1079 citations.

Not measured Topic Match

Abstract: Testing involves examining the behaviour of a system in order to discover potential faults. Given an input for a system, the challenge of distinguishing the corresponding desired, correct behaviour from potentially incorrect behavior is called the "test oracle problem". Test oracle automation is important to remove a current bottleneck that inhibits greater overall test automation. Without test oracle automation, the human has to determine whether observed behaviour is correct. The literature on test oracles has introduced techniques for oracle automation, including modelling, specifications, contract-driven development and metamorphic testing. When none of these is completely adequate, the final source of test...

[142] [On Testing Non-Testable Programs](#)

E. Weyuker. Comput. J., 1982. 643 citations.

Not measured Topic Match

No summary or abstract available

[143] [Testing and Validating Machine Learning Classifiers by Metamorphic Testing](#)

Xiaoyuan Xie, ..., and T. Chen. The Journal of systems and software, 2011. 398 citations.

Not measured Topic Match

Abstract: Machine Learning algorithms have provided core functionality to many application domains - such as bioinformatics, computational linguistics, etc. However, it is difficult to detect faults in such applications because often there is no "test oracle" to verify the correctness of the computed outputs. To help address the software quality, in this paper we present a technique for testing the implementations of machine learning classification algorithms which support such applications. Our approach is based on the technique "metamorphic testing", which has been shown to be effective to alleviate the oracle problem. Also presented include a case study on a real-world machine...

[144] [Evolutionary improvement of assertion oracles](#)

Valerio Terragni, ..., and M. Pezzè. Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2020. 55 citations.

Not measured Topic Match

Abstract: Assertion oracles are executable boolean expressions placed inside the program that should pass (return true) for all correct executions and fail (return false) for all incorrect executions. Because designing perfect assertion oracles is difficult, assertions often fail to distinguish between correct and incorrect executions. In other words, they are prone to false positives and false negatives. In this paper, we propose GAssert (Genetic ASSERTion improvement), the first technique to automatically improve assertion oracles. Given an assertion oracle and evidence of false positives and false negatives, GAssert implements a novel co-evolutionary algorithm that explores the space of possible assertions to identify...

[145] [Metamorphic Testing for Software Quality Assessment: A Study of Search Engines](#)

Zhi Quan Zhou, ..., and T. Chen. IEEE Transactions on Software Engineering, 2016. 159 citations.

Not measured Topic Match

No summary or abstract available

[146] [Compiler validation via equivalence modulo inputs](#)

Vu Le, ..., and Z. Su. Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation, 2014. 581 citations.

Not measured Topic Match

No summary or abstract available

[147] [Leveraging mutants for automatic prediction of metamorphic relations using machine learning](#)

Aravind Nair, ..., and Sigrid Eldh. Proceedings of the 3rd ACM SIGSOFT International Workshop on Machine Learning Techniques for Software Quality Evaluation, 2019. 28 citations.

Not measured Topic Match

Abstract: An oracle is used in software testing to derive the verdict (pass/fail) for a test case. Lack of precise test oracles is one of the major problems in software testing which can hinder judgements about quality. Metamorphic testing is an emerging technique which solves both the oracle problem and the test case generation problem by testing special forms of software requirements known as metamorphic requirements. However, manually deriving the metamorphic requirements for a given program requires a high level of domain expertise, is labor intensive and error prone. As an alternative, we consider the problem of automatic detection of metamorphic...

[148] [A Metamorphic Testing Approach for Online Testing of Service-Oriented Software Applications](#)

W. Chan, ..., and Karl R. P. H. Leung. Int. J. Web Serv. Res., 2007. 133 citations.

Not measured Topic Match

No summary or abstract available

[149] [On effective testing of health care simulation software](#)

Christian Murphy, ..., and L. Osterweil. Unknown journal, 2011. 36 citations.

Not measured Topic Match

No summary or abstract available

[150] [An innovative approach for testing bioinformatics programs using metamorphic testing](#)

T. Chen, ..., and Xiaoyuan Xie. BMC Bioinformatics, 2009. 167 citations.

Not measured Topic Match

Abstract: BackgroundRecent advances in experimental and computational technologies have fueled the development of many sophisticated bioinformatics programs. The correctness of such programs is crucial as incorrectly computed results may lead to wrong biological conclusion or misguide downstream experimentation. Common software testing procedures involve executing the target program with a set of test inputs and then verifying the correctness of the test outputs. However, due to the complexity of many bioinformatics programs, it is often difficult to verify the correctness of the test outputs. Therefore our ability to perform systematic software testing is greatly hindered. ResultsWe propose to use a novel software testing...

[151] [Metamorphic slice: An application in spectrum-based fault localization](#)

Xiaoyuan Xie, ..., and Baowen Xu. Inf. Softw. Technol., 2013. 110 citations.

Not measured Topic Match

No summary or abstract available

[152] [Testing Web Enabled Simulation at Scale Using Metamorphic Testing](#)

John Ahlgren, ..., and Justin Spahr-Summers. 2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), 2021. 61 citations.

Not measured Topic Match

Abstract: We report on Facebook's deployment of MIA (Metamorphic Interaction Automaton). MIA is used to test Facebook's Web Enabled Simulation, built on a web infrastructure of hundreds of millions of lines of code. MIA tackles the twin problems of test flakiness and the unknowable oracle problem. It uses metamorphic testing to automate continuous integration and regression test execution. MIA also plays the role of a test bot, automatically commenting on all relevant changes submitted for code review. It currently uses a suite of over 40 metamorphic test cases. Even at this extreme scale, a non-trivial metamorphic test suite subset yields outcomes...

[153] [Automatic system testing of programs without test oracles](#)

Christian Murphy, ..., and Gail E Kaiser. Unknown journal, 2009. 100 citations.

Not measured Topic Match

No summary or abstract available

[154] [A metamorphic relation recommendation method utilizing program syntax and semantic similarity](#)

Yin Zhao, ..., and Tao Yu. Journal of King Saud University Computer and Information Sciences, 2026. 0 citations.

Not measured Topic Match

No summary or abstract available

[155] [SEDMR: A spreadsheet error detection approach based on metamorphic testing](#)

Bo Yang, ..., and Jiatong Ma. Inf. Softw. Technol., 2026. 1 citations.

Not measured Topic Match

No summary or abstract available

[156] [Cross-Domain Multi-Label Prediction of Metamorphic Relation Patterns Leveraging Multimodal Features](#)

Zhenqiu Li, ..., and Yunwei Dong. Journal of Electronic Testing, 2025. 0 citations.

Not measured Topic Match

No summary or abstract available

[157] [Identifying metamorphic relations A data mutation directed approach](#)

Chang-ai Sun, ..., and K. Chan. Journal Not Provided, Unknown year. 0 citations.

Not measured Topic Match

No summary or abstract available

[158] [The Future of AI-Driven Software Engineering](#)

Valerio Terragni, ..., and Kelly Blincoe. ACM Transactions on Software Engineering and Methodology, 2024. 57 citations.

Not measured Topic Match

Abstract: A paradigm shift is underway in Software Engineering, with AI systems such as LLMs playing an increasingly important role in boosting software development productivity. This trend is anticipated to persist. In the next years, we expect a growing symbiotic partnership between human software developers and AI. The Software Engineering research community cannot afford to overlook this trend; we must address the key research challenges posed by the integration of AI into the software development process. In this article, we present our vision of the future of software development in an AI-driven world and explore the key challenges that our research...

[159] [Boosting Multimode Ruling in DHR Architecture With Metamorphic Relations](#)

Ruosi Li, ..., and Fan Zhang. Software Testing, 2024. 1 citations.

Not measured Topic Match

Abstract: The DHR architecture provides a revolutionary security defense structure for cyberspace. The multimode ruling in DHR is expected to alleviate the oracle problem, which still suffers from the existence of common model vulnerability. In this work, we design a test segmentation method to transform multimode ruling to a metamorphic testing problem. The text test input that causes inconsistency of heterogeneous executors is converted to a condition set, and we extract subsets of conditions based on its syntax tree. The original test can exploit a specific vulnerability, the follow up tests are composed by different subsets of conditions within the original test...

[160] [Can Metamorphic Relations be Automatically Discovered?](#)

Congying Xu and cxubl. ArXiv, 2023. 1 citations.

Not measured Topic Match

No summary or abstract available

[161] [Using Category Partition to Detect Metamorphic Relations](#)

Saba Pedram and Y. Labiche. Proceedings of the 9th ACM International Workshop on Metamorphic Testing, 2024. 1 citations.

Not measured Topic Match

Abstract: The Category Partition (CP) functional testing method has proven to be useful in various contexts. It begins by identifying parameters and environment conditions on the basis of the function's behaviour. The characteristics/categories of these parameters/environment conditions are identified and partitioned into choices. The choices of a category are mutually exclusive and can be based on input partitioning and boundary value analysis. Thereafter, the choices are combined on the basis of a selection criterion to form test frames. Once input values satisfying the conditions of a test frame's choices are identified, one is equipped with a test case. This paper suggests...

[162] [On the effectiveness of testing sentiment analysis systems with metamorphic testing](#)

Mingyue Jiang, ..., and Shuai Wang. Inf. Softw. Technol., 2022. 19 citations.

Not measured Topic Match

No summary or abstract available

[163] [Fuzzing deep-learning libraries via automated relational API inference](#)

Yinlin Deng, ..., and Lingming Zhang. Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2022. 101 citations.

Not measured Topic Match

Abstract: Deep Learning (DL) has gained wide attention in recent years. Meanwhile, bugs in DL systems can lead to serious consequences, and may even threaten human lives. As a result, a growing body of research has been dedicated to DL model testing. However, there is still limited work on testing DL libraries, e.g., PyTorch and TensorFlow, which serve as the foundations for building, training, and running DL models. Prior work on fuzzing DL libraries can only generate tests for APIs which have been invoked by documentation examples, developer tests, or DL models, leaving a large number of APIs untested. In this...

[164] [Predicting Metamorphic Relations for Matrix Calculation Programs](#)

Karishma Rahman and Upulee Kanewala. 2018 IEEE/ACM 3rd International Workshop on Metamorphic Testing (MET), 2018. 18 citations.

Not measured Topic Match

No summary or abstract available

[165] [Test Oracle Automation in the Era of LLMs](#)

F. Molina, ..., and Marcelo d'Amorim. ACM Transactions on Software Engineering and Methodology, 2024. 34 citations.

Not measured Topic Match

Abstract: The effectiveness of a test suite in detecting faults highly depends on the quality of its test oracles. Large Language Models (LLMs) have demonstrated remarkable proficiency in tackling diverse software testing tasks. This article aims to present a roadmap for future research on the use of LLMs for test oracle automation. We discuss the progress made in the field of test oracle automation before the introduction of LLMs, identifying the main limitations and weaknesses of existing techniques. Additionally, we discuss recent studies on the use of LLMs for this task, highlighting the main challenges that arise from their use, e.g.,...